USHARAMA
COLLEGE OF ENGINEERING AND TECHNOLOGY

AUTONOMOUS

On NH-16, Telaprolu, Near Gannavaram, Krishna District-521109

PH: 91777 12255, 99407 12255, 0866-2527558 www.usharama.edu.in

Department of Computer Science and Engineering

Computer Network Lab

Name of Faculty : P Bhagyasri

Subject : CN LAB

Class/Sem :III Year/I Sem

Section :A&B

Academic Year :2025-2026

USHARAMA
COLLEGE OF ENGINEERING AND TECHNOLOGY

AUTONOMOUS

Vision of the Institute

To emerge as a Centre of excellence in technical education by imparting quality teaching learning practices and research for the transformation of society.

Mission of the Institute

M1: Provide an ideal and the best class infrastructure to foster exploration in engineering and research

M2: Build dedicated faculty with student centric teaching, incorporating experiential, innovative skills.

M3: Encourage life-long learning, entrepreneurial thinking, and ethical responsibility in students to address societal challenges.

USHARAMA COLLEGE OF ENGINEERING AND TECHNOLOGY

AUTONOMOUS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department

To emerge as a skilled Technocrats on global scale in Computer Science and Engineering through quality education, innovation, collaborative researchers and entrepreneurs with moral values.

Mission of the Department

M1: To impart quality education to the students.

M2: To pursue creative research and new technologies in Computer Science and Engineering.

M3: To encourage entrepreneurship skills among students and inculcating moral and ethical values to serve for the society.

Department Program Educational Objectives (PEOs)

PEO1: Our graduates will establish themselves as effective professionals in industry, academia and entrepreneurship.

PEO2: Our graduates will become profound researchers in multiple domains.

PEO3: Our graduates will act as a leaders in society.

Department Program Specific Outcomes (PSOs)

PSO1: Illustrating a comprehensive understanding of fundamental computer system principles encompassing both hardware and software components to cultivate strong conceptual skills in processing and assigning computation solutions.

PSO2: Demonstrate and design proficient and technical abilities in algorithms, networking, web design, cloud computing and data analytics enabling the development of innovative solutions to complex real-world problems while identifying and addressing emerging research gaps

USHARAMA
COLLEGE OF ENGINEERING AND TECHNOLOGY

AUTONOMOUS

Program Outcomes (POs)

PO1: **Engineering Knowledge**: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: **Problem Analysis**: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: **Design/Development of Solutions**: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: **Conduct Investigations of Complex Problems**: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: **Engineering Tool Usage**: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: **The Engineer and The World**: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: **Ethics**: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: **Individual and Collaborative Team work**: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: **Communication**: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: **Project Management and Finance**: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: **Life-Long Learning**: Recognize the need for and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

III Year I Semester	COMPUTER NETWORKS LAB	L	T	P	C
		0	0	3	1.5

Course Objectives:

Learn basic concepts of computer networking and acquire practical notions of protocols with the emphasis on TCP/IP. A lab provides a practical approach to Ethernet/Internet networking: networks are assembled, and experiments are made to understand the layered architecture and how do some important protocols work

List of Experiments:

1. Study of Network devices in detail and connect the computers in Local Area Network.
2. Write a Program to implement the data link layer farming methods such as i) Character stuffing ii) bit stuffing.
3. Write a Program to implement data link layer farming method checksum.
4. Write a program for Hamming Code generation for error detection and correction.
5. Write a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16 and CRC CCIP.
6. Write a Program to implement Sliding window protocol for Goback N.
7. Write a Program to implement Sliding window protocol for Selective repeat.
8. Write a Program to implement Stop and Wait Protocol.
9. Write a program for congestion control using leaky bucket algorithm
10. Write a Program to implement Dijkstra‘s algorithm to compute the Shortest path through a graph.
11. Write a Program to implement Distance vector routing algorithm by obtaining routing table at each node (Take an example subnet graph with weights indicating delay between nodes).
12. Write a Program to implement Broadcast tree by taking subnet of hosts.
13. Wireshark
    i. Packet Capture Using Wire shark
    ii. Starting Wire shark
    iii. Viewing Captured Traffic
    iv. Analysis and Statistics & Filters.
14. How to run Nmap scan
15. Operating System Detection using Nmap
16. Do the following using NS2 Simulator
    i. NS2 Simulator-Introduction
    ii. Simulate to Find the Number of Packets Dropped
    iii. Simulate to Find the Number of Packets Dropped by TCP/UDP
    iv. Simulate to Find the Number of Packets Dropped due to Congestion
    v. Simulate to Compare Data Rate& Throughput.

ADD ON:

1. Write a program to implement the RSA algorithm for encryption and decryption.
2. Write a program to calculate subnet mask, number of hosts, and network address for a given IP and subnet.

Course Outcomes (COs)

CO1: Identify and demonstrate various network devices and establish a Local Area Network (LAN).
CO2: Implement and test data link layer error detection and correction techniques.
CO3: Develop and simulate flow and congestion control mechanisms using network protocols.
CO4: Apply routing algorithms and network analysis tools (Wireshark, Nmap, NS2) to analyze network performance.

CO–PO Mapping Table

Course Outcomes (COs)	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	1	1	3	1	1	1	1	1	1
CO2	2	3	3	2	2	1	1	1	1	1	1
CO3	2	3	3	3	2	1	1	1	1	1	1
CO4	2	2	2	3	3	1	1	1	1	2	2

Mapping Scale:

Scale	Meaning
3	High Correlation
2	Medium Correlation
1	Low Correlation

EXERCISE-1(A)

AIM:

1. (a) Study of following Network Devices in Detail
   - Repeater
   - Hub
   - Switch
   - Bridge
   - Router
   - Gate Way

.

Repeater:

Functioning at Physical Layer. A **repeater** is an electronic device that receives asignal and retransmits it at a higher level and/or higher power, or onto the other side of an obstruction, so that the signal can cover longer distances. Repeater have two ports ,so cannotbe use to connect for more than two devices

Important features of a repeater are as follows:

1. A repeater connects different segments of a LAN
2. A repeater forwards every frame it receives
3. A repeater is a regenerator, not an amplifier
4. It can be used to create a single extended LAN

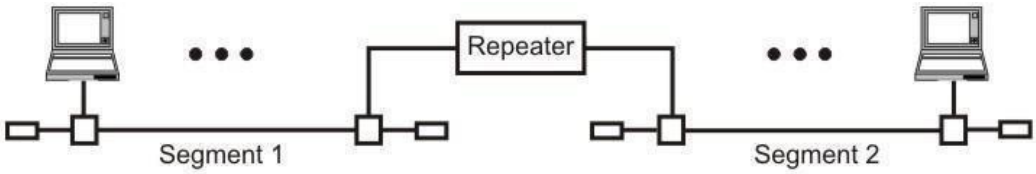


Figure Repeater connecting two LAN segments

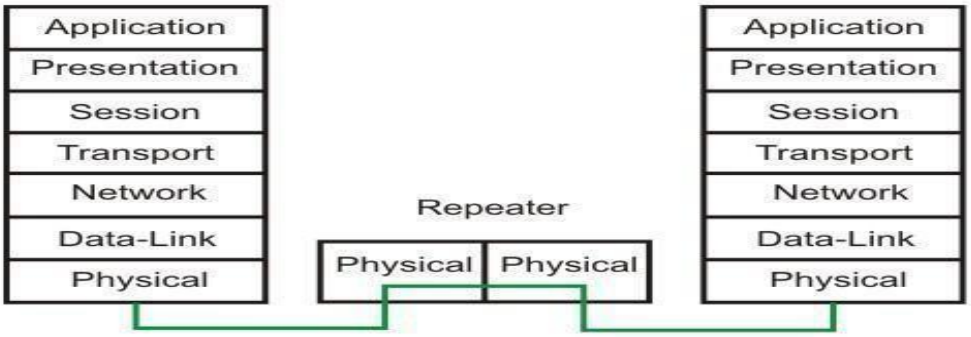


Figure Operation of a repeater as a level-1 relay

Hub:
An Ethernet hub, active hub, network hub, repeater hub, hub or concentrator is a device for connecting multiple twisted pair or fiber optic Ethernet devices together and making them act as a single network segment. Hubs work at the physical layer (layer 1) of theOSI model. The device is a form of multiport repeater. Repeater hubs also participate in collision detection, forwarding a jam signal to all ports if it detects a collision.

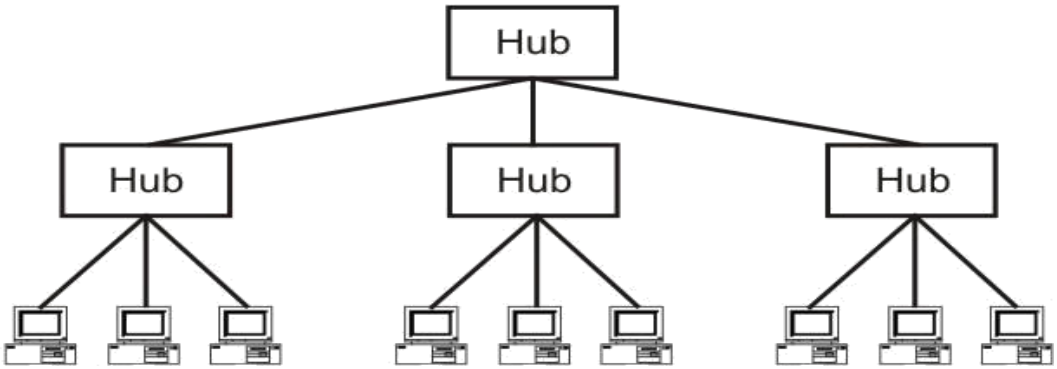
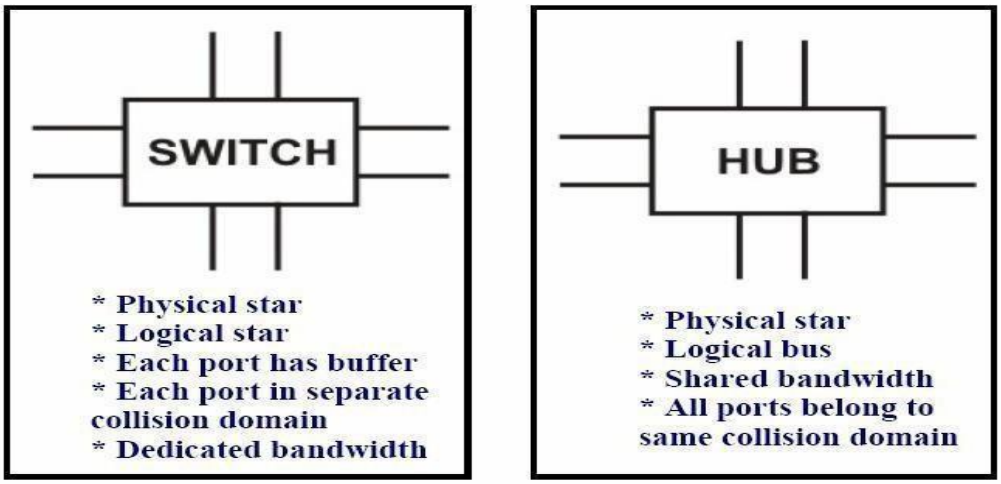


Figure: Hub as a multi-port repeater can be connected in a hierarchical manner to form a single LAN with many nodes

Switch:
A **network switch** or **switching hub** is a computer networking device that connects network segments. The term commonly refers to a network bridge that processes androutes data at the data link layer (layer 2) of the OSI model. Switches that additionally process data at the network layer (layer 3 and above) are often referred to as Layer 3 switchesor multilayer switches.



: Difference between a switch and a bridge

Some of important functionalities are:

1. Ports are provided with buffer
2. Switch maintains a directory: #address - port#
3. Each frame is forwarded after examining the #address and forwarded to the proper port#

Bridge:

A **network bridge** connects multiple network segments at the data link layer (Layer 2) of the OSI model. In Ethernet networks, the term *bridge* formally means a device that behaves according to the IEEE 802.1 D standards. A bridge and switch are very much alike; a switch being a bridge with numerous ports. *Switch* or *Layer 2 switch* is often used interchangeably with *bridge*. Bridges can analyze incoming data packets to determine if the bridge is able to send the given packet to another segment of the network.

Key features of a bridge are mentioned below:

1. A bridge operates both in physical and data-link layer
2. A bridge uses a table for filtering/routing
3. A bridge does not change the physical (MAC) addresses in a frame

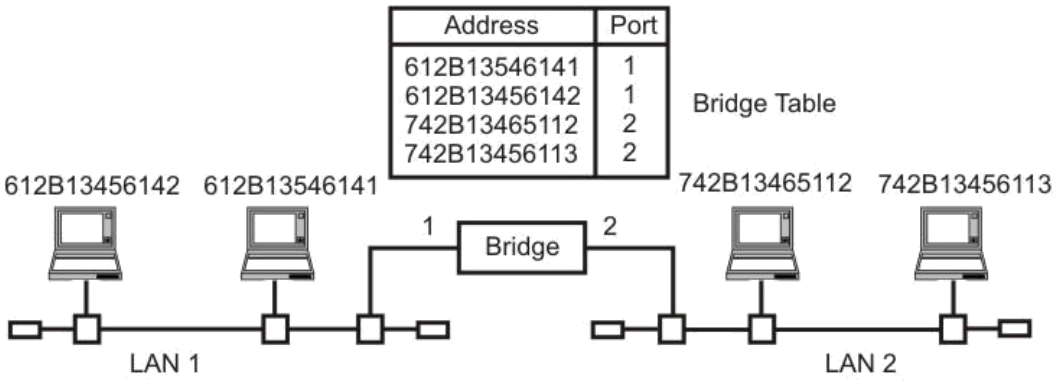


Figure: A bridge connecting two separate LANs

Types of bridges:

i) Transparent Bridges
ii) Source routing bridges

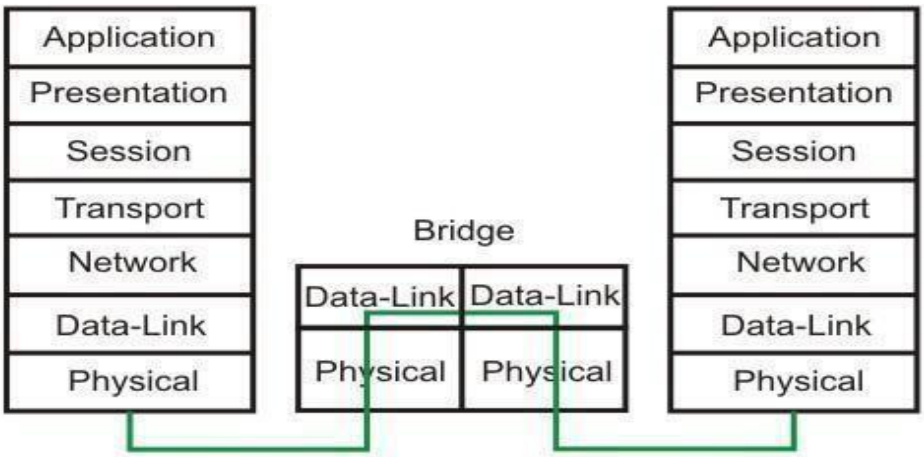


Figure Information flow through a bridge

Router:

A **router** is an electronic device that interconnects two or more computer networks, and selectively interchanges packets of data between them. Each data packet contains address information that a router can use to determine if the source and destination are on the same network, or if the data packet must be transferred from one network to another. Where multiple routers are used in a large collection of interconnected networks, the routers exchange information about target system addresses, so that each router can build upa table showing the preferred paths between any two systems on the interconnected networks.

There are two types of routers:

I. **Static routers -** Are configured manually and route data packets based oninformation in a router table.
II. **Dynamic routers -** Use dynamic routing algorithms.

There are two typesof algorithms:

1) Distance vector routing
2) Link state routing

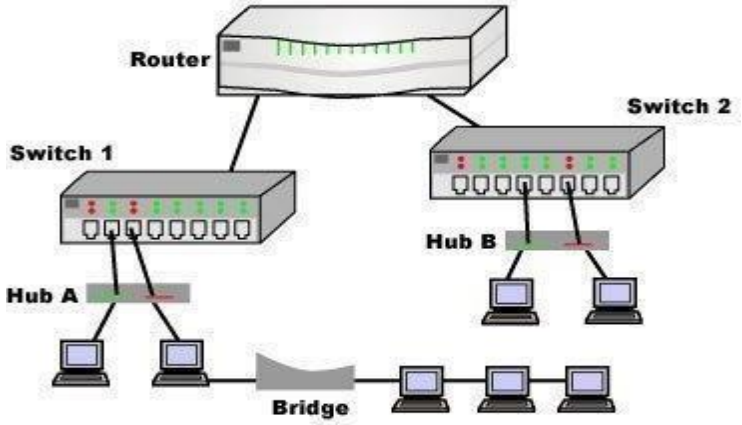
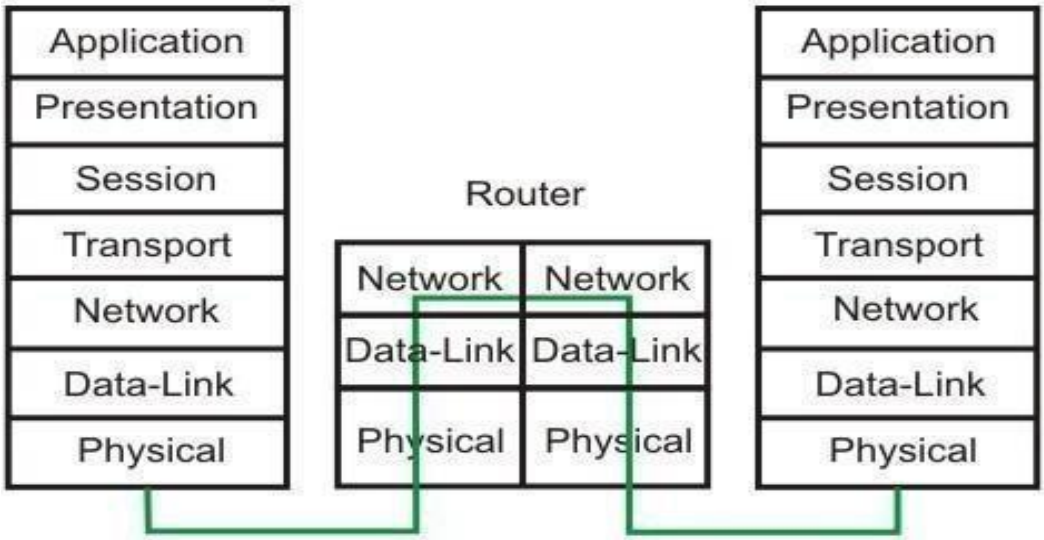


Figure: Communication through a router



Gate Way: A gateway, as the name suggests, is a passage to connect twonetworks together that may work upon different networking models.
They basically works as the messenger agents that take data from one system, interpret it, and transfer it to another system. Gateways are also called protocol converters and can operate at any network layer. Gateways are generally more complex than switch or router.

A gateway can translate information between different network data formats or network architectures.

- A gateway may contain devices such as protocol translators, impedance matching devices, rate converters, fault isolators, or signal translators as necessary to provide system interoperability. It also requires the establishment of mutually acceptable administrative procedures between both networks.

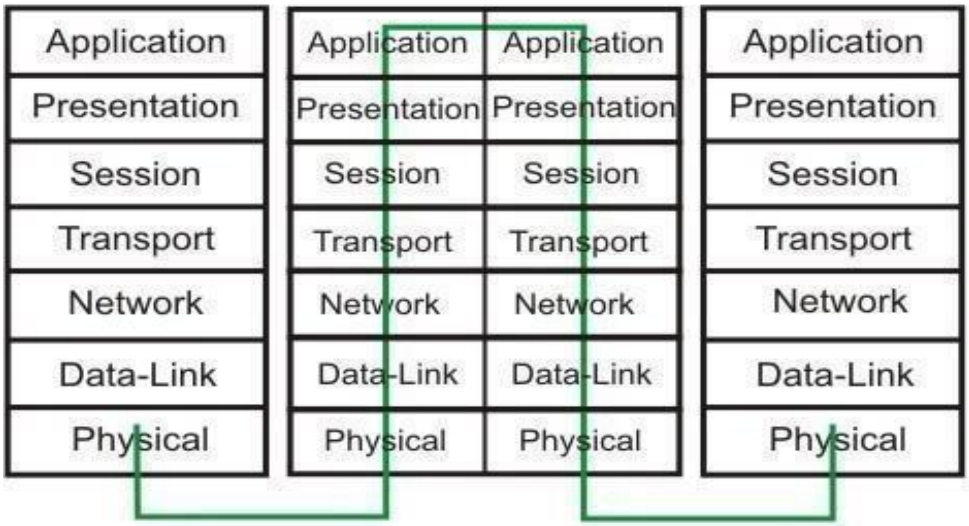


Figure: Communication through a gateway

EXERCISE-1(B)

Aim:

1. (b) Connect the computers in Local area Network.

PROCEDURE:

1. on the host computer:

follow these steps to share the internet connection:

1. Log on to the host computer as administrator or as owner.
2. click start, and then click control panel.
3. click Network and Internet connections.
4. click Network Connections.
5. Right click the connection that you use to connect to the Internet.
6. click properties.
7. Click the advanced tab.
8. under internet connection sharing, select the Allow other network users to connect through this computer's internet connection check bon
9. If you are sharing a dial-up Internet Connection whenever a computer on my network attempts to access the Internet checkbox if you want to Permit your computer to automatically connect to the Internet.
10. click ok.
11. click yes.

2. On the client computer:

To confirm the LAN adapter IP Configuration, follow these steps:

1. Log on to the client computer as Administrator or as Owner.
2. Click start, then click control panel.
3. click Network and Internet connections.
4. click Network connections.
5. Right-click Local area connection and then click properties.
6. click the General tab, Click Internet protocol (TCP / Ip) in the connection uses the following items List, and then click properties.
7. In the Internet protocol (TCP / IP) properties dialog box, click obtain an IP address automatically (if it is not already selected), and then dick ok.
8. IP Address 192.168.31.202
9. subnet mask 255.255.255.0
10. Default gateway 192.168.31. 1
11. In the Local area correction properties dialog box, click ok.
12. Quit control panel.

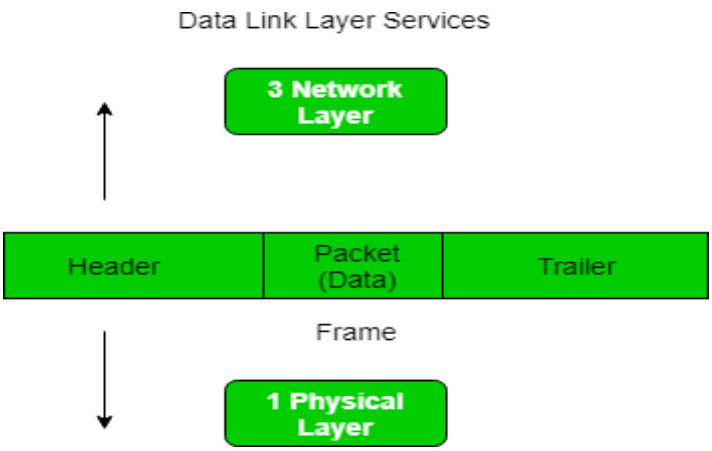
EXERCISE-2

AIM:

2. write a program to implement the data link layer framing methods such as
   i) Byte stuffing or character stuffing.
   ii) Bit stuffing

THEORY:

- In the physical layer, data transmission in the form of bits from the source to the destination.
- The data link layer packs these bits into frames.
- Data-link layer takes the packets from the Network Layer and encapsulates them into frames.
- If the frame size becomes too large, then the packet may be divided into small sized frames.
- Smaller sized frames make flow control and error control more efficient.
- Then, it sends each frame bit-by-bit on the hardware.
- At receiver's end, data link layer picks up signals from hardware and assembles them into frames.



Types of framing:

There are two types of framing:

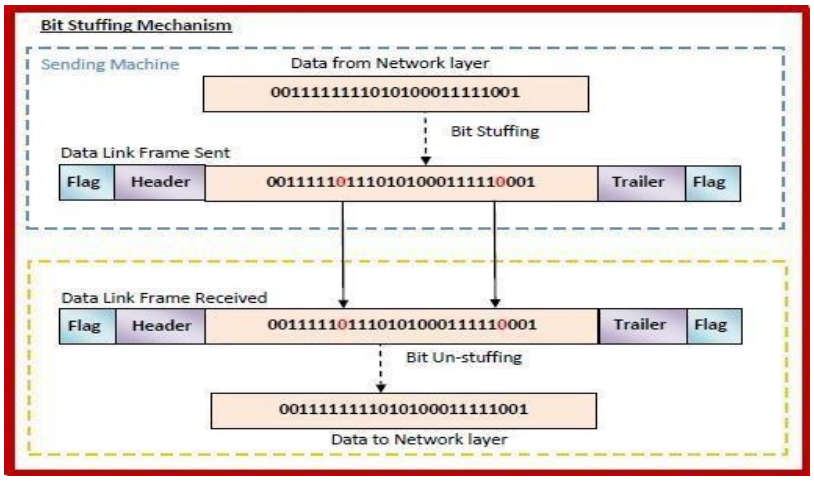
1. **Fixed-size:**
   The frame is of fixed size and there is no need to provide boundaries to the frame, the length of the frame itself acts as a delimiter.
2. **Variable size:**
   In this, there is a need to define the end of the frame as well as the beginning of the next frame to distinguish.
   It is used in local area networks.

This can be done in two ways:

- ➔ Bit Stuffing
- ➔ Byte stuffing

❖ **Bit Stuffing Mechanism:**

- Protocols use the 8-bit pattern 01111110 as flag.
- In order to differentiate the message from the flag in case of same sequence, a single bit is stuffed in the message.
- Whenever a 0 bit is followed by five consecutive 1bits in the message, an extra 0 bit is stuffed at the end of the five 1s.
- When the receiver receives the message, it removes the stuffed 0s after each sequence of five 1s.
- The un-stuffed message is then sent to the upper layers.



❖ **Byte stuffing or character stuffing:**

➔ Also referred to as character stuffing.

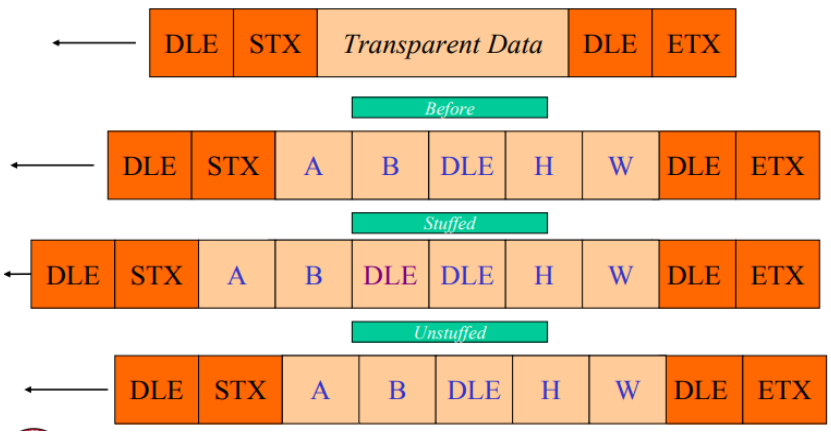
➔ ASCII characters are used as framing delimiters (e.g. DLE STX and DLE ETX) •

➔ The problem occurs when these character patterns occur within the "transparent" data.

➔ **Solution:**
sender stuffs an extra DLE or (user defined data) into the data stream just before each occurrence of an "accidental" DLE in the data stream.

➔ The data link layer on the receiving end unstuffs the DLE before giving the data to the network layer.

Example for byte stuffing:



AIM 2(A):
Write a Program to implement the data link layer farming methods such as Character stuffing.

PROGRAM:

```
#include<stdio.h>
#include<string.h>
#include<process.h>
void main()
{
int i=0,j=0,n,pos;
char a[20],b[50],ch;
printf("Enter string\n");
scanf("%s",&a);
n=strlen(a);
printf("Enter position\n");
scanf("%d",&pos);
if(pos>n)
{
printf("invalid position, Enter again :");
scanf("%d",&pos);}
printf("Enter the character\n");
ch=getche();
b[0]='d';
b[1]='l';
b[2]='e';
b[3]='s';
b[4]='t';
b[5]='x';
j=6;
while(i<n)
{
if(i==pos-1)
{
b[j]='d';
b[j+1]='l';
b[j+2]='e';
b[j+3]=ch;
b[j+4]='d';
b[j+5]='l';
b[j+6]='e';
j=j+7;
}
if(a[i]=='d' && a[i+1]=='l' && a[i+2]=='e')
{
b[j]='d';
b[j+1]='l';
b[j+2]='e';
j=j+3;
}
```

```
b[j]=a[i];
i++;
j++;
}
b[j]='d';
b[j+1]='l';
b[j+2]='e';
b[j+3]='e';
b[j+4]='t';
b[j+5]='x';
b[j+6]='\0';
printf("\nframe after stuffing:\n");
printf("%s",b);
}
```

OUTPUT:

Enter string: COMPUTER
Enter position: 3
Enter the character: @

Frame after stuffing:
dlestxCOdle@dleMPUTERdleetx

AIM 2(B):
Write a Program to implement the data link layer farming methods such as Bit stuffing.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
void main()
{
int i=0,count=0;
char databits[80];
printf("Enter Data Bits: ");
scanf("%s",databits);
printf("\nData Bits After Bit stuffing: ");
for(i=0; i<strlen(databits); i++)
{
if(databits[i]=='1')
count++;
else
count=0;
printf("%c",databits[i]);
if(count==5)
{
printf("0");
count=0;
}
}
getch();
}
```

OUTPUT:

Enter Data Bits: 0111110111110

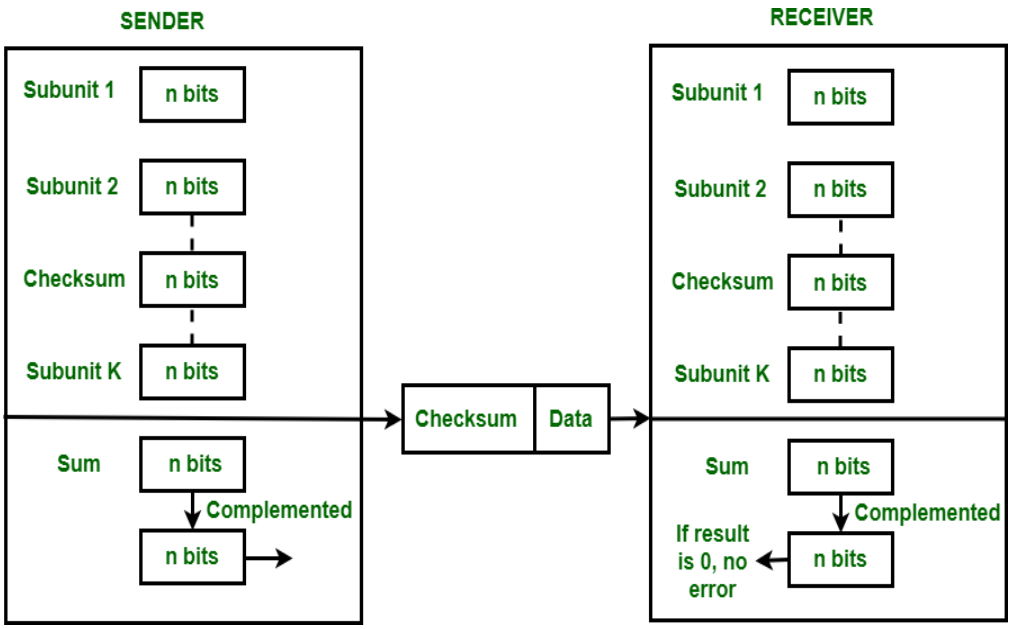
Data Bits After Bit Stuffing: 011111001111100

EXERCISE-3

AIM:
Write a Program to implement data link layer farming method checksum.

THEORY:
Checksum is the error detection method used by upper layer protocols and is considered to be more reliable than LRC, VRC and CRC.

- This method makes the use of Checksum Generator on Sender side and Checksum Checker on Receiver side.
- At the Sender side, the data is divided into equal subunits of n bit length by the checksum generator. This bit is generally of 16-bit length.
- These subunits are then added together using one's complement method. This sum is of n bits. The resultant bit is then complemented.
- This complemented sum which is called checksum is appended to the end of original data unit and is then transmitted to Receiver.



- The Receiver after receiving data + checksum passes it to checksum checker.
- Checksum checker divides this data unit into various subunits of equal length and adds all these subunits.
- These subunits also contain checksum as one of the subunits. The resultant bit is then complemented.
- If the complemented result is zero, it means the data is error-free.
- If the result is non-zero it means the data contains an error and Receiver rejects it.

Example:
If the data unit to be transmitted is 10101001 00111001, the following procedure is used at Sender side and Receiver side.

Sender Side:

10101001	subunit 1
00111001	subunit 2
11100010	sum (using 1s complement)
00011101	checksum (complement of sum)

Data transmitted to Receiver is –

1010001 00111001	00011101
Data	Checksum

Receiver Side :

10101001	subunit 1
00111001	subunit 2
00011101	checksum
11111111	sum
00000000	sums' complement

Result is zero, it means no error.

Advantage:

- The checksum detects all the errors involving an odd number of bits as well as the error involving an even number of bits.

Disadvantage:

- The main problem is that the error goes undetected if one or more bits of a subunit is damaged and the corresponding bit or bits of a subunit are damaged and the corresponding bit or bits of opposite value in second subunit are also damaged. This is because the sum of those columns remains unchanged.

PROGRAM:

```
#include<stdio.h>

int add(int, int);
int com(int);

int main() {
   int i, dl;
   int data1[10], data2[10], newdata[10], comp[10], checksum[10];

   printf("\nEnter the data length = ");
   scanf("%d", &dl);

   printf("\nEnter the frame1 or data1 in the form of 0's and 1's:\n");
```

```
for (i = 0; i < dl; i++)
   scanf("%d", &data1[i]);

printf("\nEnter the frame2 or data2 in the form of 0's and 1's:\n");
for (i = 0; i < dl; i++)
   scanf("%d", &data2[i]);

// Addition of data1 and data2
for (i = dl - 1; i >= 0; i--) {
   newdata[i] = add(data1[i], data2[i]);
}

printf("\n\nData 1 : ");
for (i = 0; i < dl; i++)
   printf("%d", data1[i]);

printf("\nData 2 : ");
for (i = 0; i < dl; i++)
   printf("%d", data2[i]);

printf("\n\nThe new data is : ");
for (i = 0; i < dl; i++) {
   printf("%d", newdata[i]);
}

printf("\nChecksum : ");
for (i = 0; i < dl; i++) {
   checksum[i] = com(newdata[i]);
   printf("%d", checksum[i]);
}

// Receiver side
printf("\n\nReceiver Side : \n");
printf("\nData : ");
for (i = 0; i < dl; i++)
   printf("%d", data1[i]);
printf(" ");
for (i = 0; i < dl; i++)
   printf("%d", data2[i]);
printf(" ");
for (i = 0; i < dl; i++)
   printf("%d", checksum[i]);

// Addition at receiver
printf("\nAddition : ");
for (i = dl - 1; i >= 0; i--) {
   newdata[i] = add(newdata[i], checksum[i]);
}
for (i = 0; i < dl; i++) {
   printf("%d", newdata[i]);
}
```

```
  // Complement at receiver
   printf("\nComplement : ");
   for (i = 0; i < dl; i++) {
      comp[i] = com(newdata[i]);
      printf("%d", comp[i]);
   }

   printf("\n");
   return 0;
}

int add(int x, int y) {
   static int carry = 0;
   if (x == 1 && y == 1 && carry == 0) {
      carry = 1; return 0;
   } else if (x == 1 && y == 1 && carry == 1) {
      carry = 1; return 1;
   } else if (x == 1 && y == 0 && carry == 0) {
      carry = 0; return 1;
   } else if (x == 1 && y == 0 && carry == 1) {
      carry = 1; return 0;
   } else if (x == 0 && y == 1 && carry == 0) {
      carry = 0; return 1;
   } else if (x == 0 && y == 1 && carry == 1) {
      carry = 1; return 0;
   } else if (x == 0 && y == 0 && carry == 0) {
      carry = 0; return 0;
   } else {
      carry = 0; return 1;
   }
}

int com(int a) {
   if (a == 0)
      return 1;
   else
      return 0;
}}
```

OUTPUT:

Enter the data length= 4

Enter the frame1 or data1 in the form of 0's and 1's:1100

Enter the frame2 or data2 in the form of 0's and 1's:0101

Data 1:1100

Data 2:0101

New data is:10001

Checksum:01110

Receiver side:

Data:1100 0101 0110

Addition:11111

Compliment:00000

EXERCISE-4

Write a program for Hamming Code generation for error detection and correction.

```
#include<stdio.h>
#include<conio.h>
void main() {
	int data[7],rec[7],i,c1,c2,c3,c;
	printf("this works for message of 4bits in size \nenter message bit one by one:  ");
	scanf("%d%d%d%d",&data[0],&data[1],&data[2],&data[4]);
	data[6]=data[0]^data[2]^data[4];
	data[5]=data[0]^data[1]^data[4];
	data[3]=data[0]^data[1]^data[2];
	printf("\nthe encoded bits are given below: \n");
	for (i=0;i<7;i++) {
		printf("%d ",data[i]);
	}
	printf("\nenter the received data bits one by one: ");
	for (i=0;i<7;i++) {
		scanf("%d",&rec[i]);
	}
	c1=rec[6]^rec[4]^rec[2]^rec[0];
	c2=rec[5]^rec[4]^rec[1]^rec[0];
	c3=rec[3]^rec[2]^rec[1]^rec[0];
	c=c3*4+c2*2+c1 ;
	if(c==0) {
		printf("\ncongratulations there is no error: ");
	} else {
		printf("\nerron on the postion: %d\nthe correct message is \n",c);
		if(rec[7-c]==0)
				rec[7-c]=1; else
				rec[7-c]=0;
		for (i=0;i<7;i++) {
			printf("%d ",rec[i]);
		}
	}
	getch();
}
```

OUTPUT:

```
this works for message of 4bits in size
enter message bit one by one:  1 0 1 1

the encoded bits are given below:
1 0 1 0 1 0 1
enter the received data bits one by one: 1 0 1 0 1 0 1
ERROR!

congratulations there is no error:
```

=== Code Exited With Errors ===

OUTPUT 2:

this works for message of 4bits in size
enter message bit one by one: 1 0 1 1 0 1

the encoded bits are given below:
1 0 1 0 1 0 1
enter the received data bits one by one: 1 0 1 0 1 0 1

erron on the postion: 1
the correct message is
0 1 1 0 1 0 0

EXERCISE-5

AIM:

Write a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16 and CRC CCIP.

THEORY:

1. CRC or Cyclic Redundancy Check is a method of detecting accidental changes/errors in the communication channel.
2. CRC uses Generator Polynomial which is available on both sender and receiver side.
3. An example generator polynomial is of the form like $x^3 + x + 1$. This generator polynomial represents key 1011. Another example is $x^2 + 1$ that represents key 101.

n : Number of bits in data to be sent from sender side.

k : Number of bits in the key obtained from generator polynomial.

Sender Side (Generation of Encoded Data from Data and Generator Polynomial (or Key)):

1. The binary data is first augmented by adding k-1 zeros in the end of the data
2. Use *modulo-2 binary division* to divide binary data by the key and store remainder of division.
3. Append the remainder at the end of the data to form the encoded data and send the same

Receiver Side (Check if there are errors introduced in transmission)

Perform modulo-2 division again and if the remainder is 0, then there are no errors.
In this article we will focus only on finding the remainder i.e. check word and the code word.

Modulo 2 Division:

The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. Just that instead of subtraction, we use XOR here.

- In each step, a copy of the divisor (or data) is XORed with the k bits of the dividend (or key).
- The result of the XOR operation (remainder) is (n-1) bits, which is used for the next step after 1 extra bit is pulled down to make it n bits long.
- When there are no bits left to pull down, we have a result. The (n-1)-bit remainder which is appended at the sender side.

Example 1:

Data word to be sent - 100100

Key - 1101 [Or generator polynomial $x^3 + x^2 + 1$]

Sender Side:

```
            111101
1101 | 100100000 |
       1101
        1000
        1101
          1010
          1101
           1110
           1101
            0110
            0000
             1100
             1101
              001
```

v

Therefore, the remainder is 001 and hence the encoded data sent is 100100001.

Receiver Side:

Code word received at the receiver side 100100001

```
            111101
1101 | 100100001 |
       1101
        1000
        1101
          1010
          1101
           1110
           1101
            0110
            0000
             1101
             1101
              0000
```

Therefore, the remainder is all zeros. Hence, the data received has no error.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#define N strlen(g)
char t[28],cs[28],g[28]; int a,e,c,b;
void xor()
{
for(c=1;c<N;c++) cs[c]=((cs[c]==g[c])?'0':'1');
}
void crc()
{
for(e=0;e<N;e++) cs[e]=t[e];
do
{ if(cs[0]=='1')
xor();
for(c=0;c<N-1;c++) cs[c]=cs[c+1];
cs[c]=t[e++];
}while(e<=a+N-1);
}
int main()
{
int flag=0; do{
printf("\n1.crc12\n2.crc16\ncrc ccit\n4.exit\n\nEnter your option."); scanf("%d",&b);
switch(b)
{
case1:strcpy(g,"1100000001111"); break;
case2:strcpy(g,"11000000000000101"); break;
case3:strcpy(g,"10001000000100001"); break;
case 4:return 0;
}
printf("\n enter data:"); scanf("%s",t);
printf("\n        \n");
printf("\n generating polynomial:%s",g); a=strlen(t);
for(e=a;e<a+N-1;e++) t[e]='0';
printf("\n        \n");
printf("modified data is:%s",t); printf("\n      \n");
crc();
printf("checksum is:%s",cs); for(e=a;e<a+N-1;e++) t[e]=cs[e-a];
printf("\n        \n");
printf("\n final codeword is : %s",t); printf("\n          \n");
printf("\ntest error detection 0(yes) 1(no)?:"); scanf("%d",&e);
if(e==0)
{
do{
printf("\n\tenter the position where error is to be inserted:"); scanf("%d",&e);
}
```

```
while(e==0||e>a+N-1);
t[e-1]=(t[e-1]=='0')?'1':'0'; printf("\n \n");
printf("\n\terroneous data:%s\n",t);
}
crc();
for(e=0;(e<N-1)&&(cs[e]!='1');e++); if(e<N-1)
printf("error detected\n\n"); else
printf("\n no error detected \n\n"); printf("\n  ");
}while(flag!=1);
}
crc();
for(e=0;(e<N-1)&&(cs[e]!='1');e++); if(e<N-1)
printf("error detected\n\n"); else
printf("\n no error detected \n\n"); printf("\n  ");
}while(flag!=1)
```

OUTPUT:

1. crc12
2. crc16
3. crc ccit
4. exit

Enter your option:1
enter data: 1100110011100011
----
generating polynomial: 1100000001111
----
modified data is: 11001100111000110000000000001100000001111
----
checksum is: 1101110110001
----
final codeword is: 11001100111000111101110110001100000001111
----
test error detection
0(yes) (no)?:1 no error detected
1.crc12 2.crc16 3.crc cit 4.exit
Enter your option.2
enter data:11001100111000
----
generating polynomial: 11000000000000101
----
modified data is: 11001100111000000000000000000000000000000000101

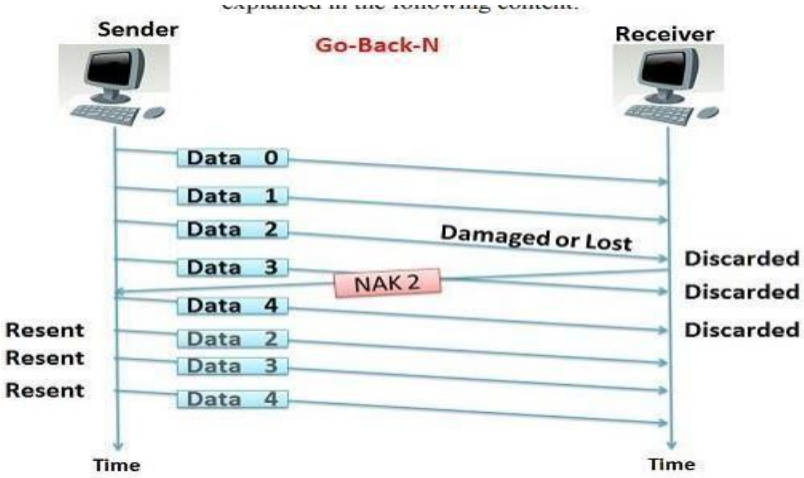
EXERCISE-6

AIM: Write a Program to implement Sliding window protocol for Go Back N ARQ.

THEORY: Error Control is a technique of error detection and retransmission.

Definition of Go-Back-N:

- Go-Back-N protocol is a sliding window protocol.
- It is a mechanism to detect and control the error in data link layer.
- During transmission of frames between sender and receiver, if a frame is damaged, lost, or an acknowledgement is lost then the action performed by sender and receiver is explained in the following content.



Damaged Frame:

- If a receiver receives a damaged frame or if an error occurs while receiving a frame then, the receiver sends the NAK (negative acknowledgement) for that frame along with that frame number, that it expects to be retransmitted.
- After sending NAK, the receiver discards all the frames that it receives, after a damaged frame.

- The receiver does not send any ACK (acknowledgement) for the discarded frames.
- After the sender receives the NAK for the damaged frame, it retransmits all the frames onwards the frame number referred by NAK.

Lost frame:

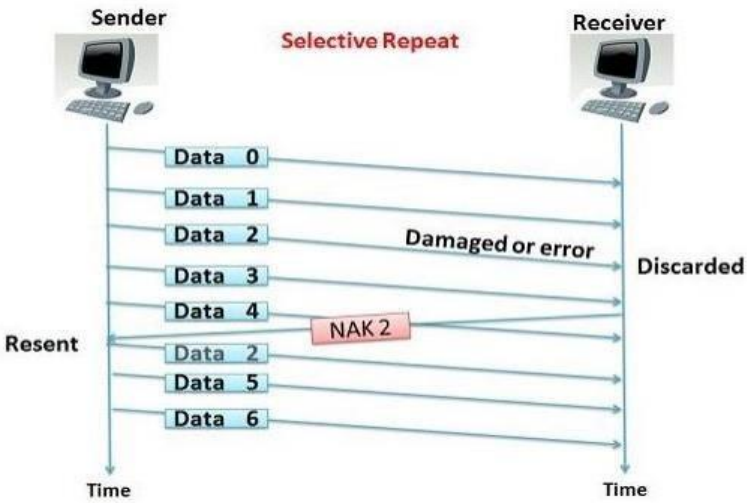
- The receiver checks the number on each frame, it receives.
- If a frame number is skipped in a sequence, then the receiver easily detects the loss of a frame as the newly received frame is received out of sequence.
- The receiver sends the NAK for the lost frame and then the receiver discards all the frames received after a lost frame.
- The receiver does not send any ACK (acknowledgement) for that discarded frames.
- After the sender receives the NAK for the lost frame, it retransmits the lost frame referred by NAK and also retransmits all the frames which it has sent after the lost frame.

Lost Acknowledgement:

- If the sender does not receive any ACK or if the ACK is lost or damaged in between the transmission.
- The sender waits for the time to run out and as the time run outs, the sender retransmits all the frames for which it has not received the ACK.
- The sender identifies the loss of ACK with the help of a timer.

Definition of Selective Repeat:

- Selective repeat is also the sliding window protocol which detects or corrects the error occurred in data link layer.
- The selective repeat protocol retransmits only that frame which is damaged or lost.
- In selective repeat protocol, the retransmitted framed is received out of sequence.
  The Selective repeat protocol can perform following actions
  - The receiver is capable of sorting the frame in a proper sequence, as it receives the retransmitted frame whose sequence is out of order of the receiving frame.
  - The sender must be capable of searching the frame for which the NAK has been received.
  - The receiver must contain the buffer to store all the previously received frame on hold till the retransmitted frame is sorted and placed in a proper sequence.
  - The ACK number, like NAK number, refers to the frame which is lost or damaged.
  - It requires the less window size as compared to go-back-n protocol.



PROGRAM:

```
#include<stdio.h>
int main()
{
        int windowsize,sent=0,ack,i;
        printf("enter window size\n");
        scanf("%d",&windowsize);
        while(1)
        {
                for( i = 0; i < windowsize; i++)
                        {
                                printf("Frame %d has been transmitted.\n",sent);
                                sent++;
                                if(sent == windowsize)
                                        break;
                        }
                        printf("\nPlease enter the last Acknowledgement received.\n");
                        scanf("%d",&ack);

                        if(ack == windowsize)
                                break;
                        else
                                sent = ack;
        }
return 0;
}
```

Output :

```
enter window size
8
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 4 has been transmitted.
Frame 5 has been transmitted.
Frame 6 has been transmitted.
Frame 7 has been transmitted.
Please enter the last Acknowledgement received.
2
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 4 has been transmitted.
Frame 5 has been transmitted.
Frame 6 has been transmitted.
Frame 7 has been transmitted.
Please enter the last Acknowledgement received.
8
```

EXERCISE-7

AIM: Write a Program to implement Sliding window protocol for Selective repeat.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<unistd.h>
int n,r;
struct frame
{
char ack;
int data;
}frm[10];
int sender(void);
void recvack(void);
void resend(void);
void selective(void);
int main()
{
int c;
do
{
printf("\n\n1.Selective repeat ARQ\n \n2.exit");
printf("\nEnter your choice:");
scanf("%d",&c);
switch(c)
{
case 1:selective();
break;
case 2:exit(0);
break;
}
}while(c>=4);
}
void selective()
{
sender();
recvack();
resend();
printf("\nAll packets sent successfully");
}
int sender()
{
int i;
printf("\nEnter the no. of packets to be sent:");
scanf("%d",&n);
for(i=1;i<=n;i++)
```

```
{
printf("\nEnter data for packets[%d]",i);
scanf("%d",&frm[i].data);
frm[i].ack='y';
}
return 0;
}
void recvack()
{
int i;
rand();
r=rand()%n;
frm[r].ack='n';
for(i=1;i<=n;i++)
{
if(frm[i].ack=='n')
printf("\nThe packet number %d is not received\n",r);
}
}
void resend() //SELECTIVE REPEAT
{
printf("\nresending packet %d",r);
sleep(2);
frm[r].ack='y';
printf("\nThe received packet is %d",frm[r].data);
}
```

OUTPUT:

1. selective repeat ARQ
2. exit

Enter your choice:1
Enter the no. of packets to be sent:3
Enter data for packets[1] 101
Enter data for packets[2] 010
Enter data for packets[3] 111
The packet number 2 is not received
resending packet 2
The received packet is 010
All packets sent successfully.

EXERCISE-8

AIM: Implementation of stop and wait protocol.

PROGRAM:

```
#include  <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>  // For sleep()

#define TOTAL_FRAMES 5

// Simulate sending a frame (with optional error simulation)
int sendFrame(int frameNumber) {
    printf("Sending Frame %d\n", frameNumber);

    // Simulate transmission delay
    sleep(1);

    // Simulate random frame loss or error (20% chance)
    if (rand() % 10 < 2) {
        printf("Frame %d lost or corrupted during transmission!\n", frameNumber);
        return 0; // Failure
    }

    return 1; // Success
}

// Simulate receiving an acknowledgment
int receiveAck(int frameNumber) {
    // Simulate delay in acknowledgment
    sleep(1);

    // Simulate random ACK loss (10% chance)
    if (rand() % 10 < 1) {
        printf("ACK for Frame %d lost!\n", frameNumber);
        return 0; // ACK lost
    }

    printf("ACK for Frame %d received.\n", frameNumber);
    return 1; // ACK received
}

int main() {
    srand(time(NULL)); // Seed random number generator

    int frameNumber = 1;
```

```
  while (frameNumber <= TOTAL_FRAMES) {
      if (sendFrame(frameNumber)) {
         // Wait for acknowledgment
         if (receiveAck(frameNumber)) {
            // ACK received, send next frame
            frameNumber++;
         } else {
            printf("Timeout waiting for ACK for Frame %d. Resending...\n", frameNumber);
            // Resend same frame
         }
      } else {
         printf("Resending Frame %d...\n", frameNumber);
         // Resend same frame
      }
   }

   printf("All frames sent and acknowledged successfully.\n");

   return 0;
}
```

output:

```
Sending Frame 1
ACK for Frame 1 received.
Sending Frame 2
ACK for Frame 2 lost!
Timeout waiting for ACK for Frame 2. Resending...
Sending Frame 2
ACK for Frame 2 received.
Sending Frame 3
ACK for Frame 3 received.
Sending Frame 4
Frame 4 lost or corrupted during transmission!
Resending Frame 4...
Sending Frame 4
ACK for Frame 4 received.
Sending Frame 5
ACK for Frame 5 received.
All frames sent and acknowledged successfully.

--------------------------------------
Process exited after 13.15 seconds with return value 0
Press any key to continue . . .
```

EXERCISE-9

AIM: Write a program for congestion control using leaky bucket algorithm.

THEORY:

Congestion:
A state occurring in network layer when the message traffic is so heavy that it slows down network response time.

Effects of Congestion:

- ➔ As delay increases, performance decreases.
- ➔ If delay increases, retransmission occurs, making situation worse.

Congestion control algorithms:

1. Congestion Control is a mechanism that controls the entry of data packets into the network, enabling a better use of a shared network infrastructure and avoiding congestive collapse.
2. Congestive-Avoidance Algorithms (CAA) are implemented at the TCP layer as the mechanism to avoid congestive collapse in a network.

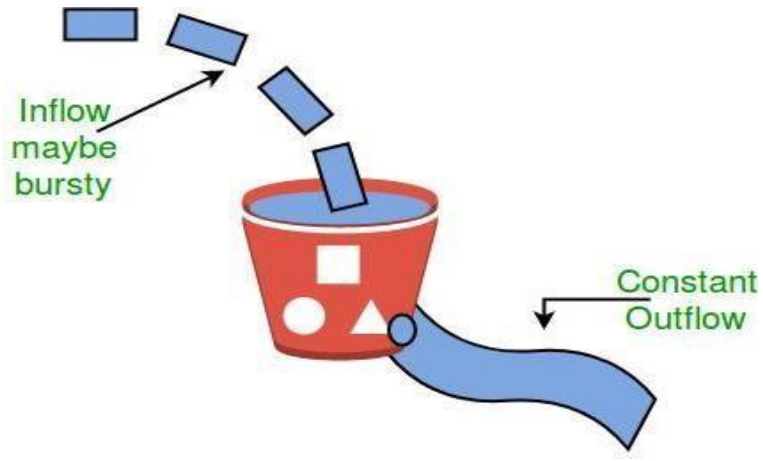
3. There are two congestion control algorithm which are as follows:
   i. Leaky Bucket Algorithm
   ii. Token bucket Algorithm

Leaky Bucket Algorithm:

- The leaky bucket algorithm discovers its use in the context of network traffic shaping or rate-limiting.
- A leaky bucket execution and a token bucket execution are predominantly used for traffic shaping algorithms.
- This algorithm is used to control the rate at which traffic is sent to the network and shape the burst traffic to a steady traffic stream.
- The disadvantages compared with the leaky-bucket algorithm are the inefficient use of available network resources.
- The large area of network resources such as bandwidth is not being used effectively.

Let us consider an example:

- Imagine a bucket with a small hole in the bottom. No matter at what rate water enters the bucket, the outflow is at constant rate.
- When the bucket is full with water additional water entering spills over the sides and is lost.



Similarly, each network interface contains a leaky bucket and the following steps are involved in leaky bucket algorithm:

1. When host wants to send packet, packet is thrown into the bucket.
2. The bucket leaks at a constant rate, meaning the network interface transmits packets at a constant rate.
3. Bursty traffic is converted to a uniform traffic by the leaky bucket.
4. In practice the bucket is a finite queue that outputs at a finite rate.

Token bucket Algorithm:

- The leaky bucket algorithm has a rigid output design at an average rate independent of the bursty traffic.
- In some applications, when large bursts arrive, the output is allowed to speed up. This calls for a more flexible algorithm, preferably one that never loses information. Therefore, a token bucket algorithm finds its uses in network traffic shaping or rate-limiting.
- It is a control algorithm that indicates when traffic should be sent. This order comes based on the display of tokens in the bucket.
- The bucket contains tokens. Each of the tokens defines a packet of predetermined size. Tokens in the bucket are deleted for the ability to share a packet.
- When tokens are shown, a flow to transmit traffic appears in the display of tokens.
- No token means no flow sends its packets. Hence, a flow transfers traffic up to its peak burst rate in good tokens in the bucket.

Need of token bucket Algorithm:

- The leaky bucket algorithm enforces output pattern at the average rate, no matter how bursty the traffic is.
- So in order to deal with the bursty traffic we need a flexible algorithm so that the data is not lost.
- One such algorithm is token bucket algorithm.

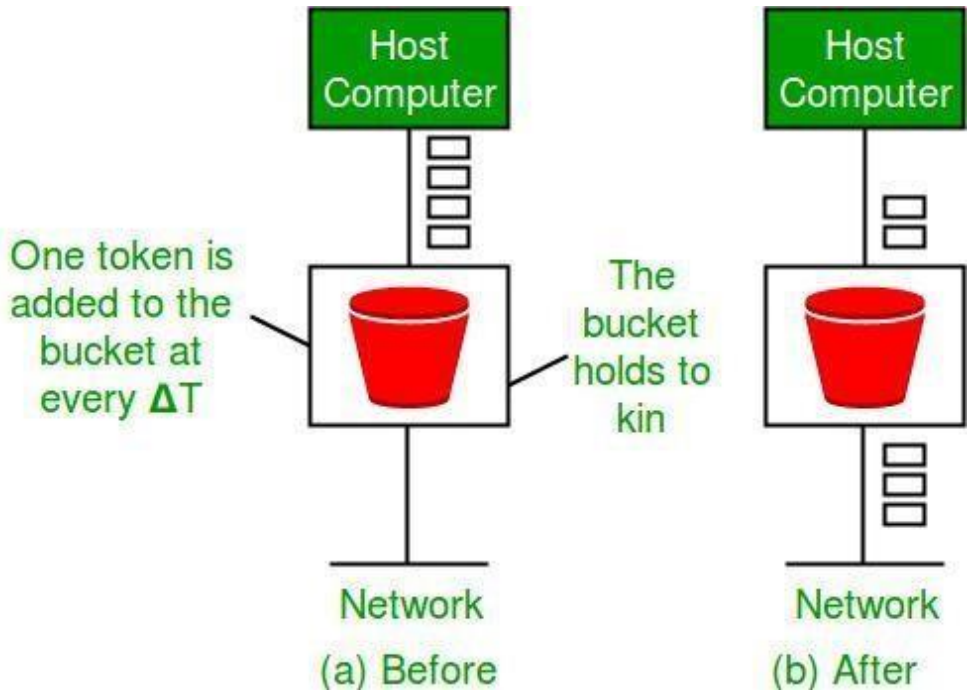
Steps of this algorithm can be described as follows:

1. In regular intervals tokens are thrown into the bucket. *f*
2. The bucket has a maximum capacity. *f*
3. If there is a ready packet, a token is removed from the bucket, and the packet is sent.
4. If there is no token in the bucket, the packet cannot be sent.

Let's understand with an example,

In figure (A) we see a bucket holding three tokens, with five packets waiting to be transmitted. For a packet to be transmitted, it must capture and destroy one token.
In figure (B) we see that three of the five packets have gotten through, but the other two are stuck waiting for more tokens to be generated.

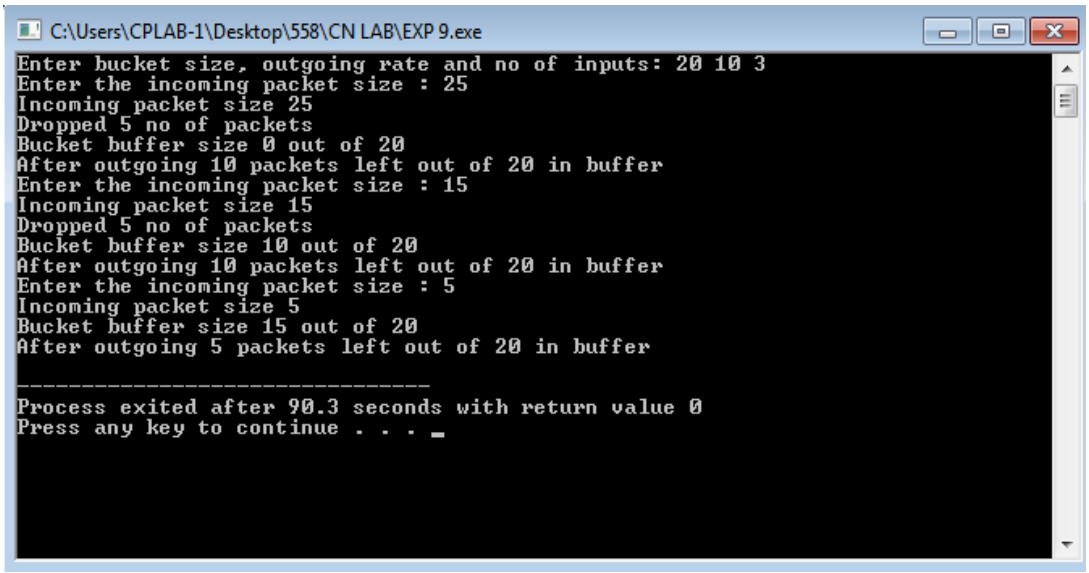
Formula: M * s = C + ρ * s where S – is time taken M – Maximum output rate ρ – Token arrival rate C – Capacity of the token bucket in byte
Let's understand with an example,



PROGRAM:

```
#include<stdio.h>
int main()
{
   int incoming, outgoing, buck_size, n, store = 0;
   printf("Enter bucket size, outgoing rate and no of inputs: ");
   scanf("%d %d %d", &buck_size, &outgoing, &n);
   while (n != 0)
   {
      printf("Enter the incoming packet size : ");
      scanf("%d", &incoming);
      printf("Incoming packet size %d\n", incoming);
      if (incoming <= (buck_size - store)){
         store += incoming;
         printf("Bucket buffer size %d out of %d\n", store, buck_size);
      }
else {
         printf("Dropped %d no of packets\n", incoming - (buck_size - store));
         printf("Bucket buffer size %d out of %d\n", store, buck_size);
         store = buck_size;
      }
      store = store - outgoing;
      printf("After outgoing %d packets left out of %d in buffer\n", store, buck_size);
      n--;
   }
}
```

OUTPUT:



EXERCISE-10

AIM: To write a program to implement Dijkstra's algorithm for finding theshortest path in a graph using the C programming language.

DESCRIPTION:
Dijkstra's algorithm finds theshortest path from a sourcenodeto all other nodes in a weighted graph. It works by repeatedly selecting the node with the minimum distance, updating the distances to its neighboring nodes, and marking it as visited. The process continues until all nodes have been visited.

PROGRAM:

```
#include  <stdio.h>
#include <limits.h>
#define V 4
int minDistance(int dist[], int visited[]) {
   int min = INT_MAX, min_index,v;
   for (v = 0; v < V; v++)
      if (!visited[v] && dist[v] < min) { min = dist[v]; min_index = v; }
   return min_index;
}

void dijkstra(int graph[V][V], int start) {
   int dist[V], visited[V] = {0},i,count,v;
   for (i = 0; i < V; i++)
         dist[i] = INT_MAX;
   dist[start] = 0;

   for (count = 0; count < V - 1; count++)
         {
      int u = minDistance(dist, visited); visited[u] = 1;
      for (v = 0; v < V; v++)
      {
         if (!visited[v] && graph[u][v] && dist[u] + graph[u][v] < dist[v])
            dist[v] = dist[u] + graph[u][v];
         }
   }

   printf("Vertex\tDistance from Source\n");
   for (i = 0; i < V; i++)
      printf("%d\t%d\n", i, dist[i]);
}

int main() {
   int graph[V][V] = {
      {0, 1, 4, 0},
      {1, 0, 2, 5},
      {4, 2, 0, 1},
      {0, 5, 1, 0}
```

```
};
dijkstra(graph, 0);
return 0;
```

Expected Output:

For the given graph and starting node 0, the expected output would look like this:

```
rajashekhar-sunkara@MiWiFi-R4A-srv:~/Documents$ gcc djikstra\'s.c -o djikstra
rajashekhar-sunkara@MiWiFi-R4A-srv:~/Documents$ ./djikstra
Vertex   Distance from Source
0                0
1                1
2                3
3                4
```

Exercise 11

AIM: Write a Program to implement Distance vector routing algorithm by obtaining routingtable at each node (Take an example subnet graph with weights indicating delay between nodes).

DESCRIPTION: Distance Vector Routing is a distributed routing protocol where each node(or router) maintains a routing table that contains thedistance(cost) to all other nodes in the network. The table is updated iteratively by exchanging information with neighboringnodes. This algorithm helps determine the shortest path to each destination node in the network basedonthe known link costs.

PROGRAM:

```
#include <stdio.h>
#define MAX_NODES 4
#define INFINITY 9999
void distanceVector(int costMatrix[MAX_NODES][MAX_NODES], int nodes) {
int distance[MAX_NODES][MAX_NODES], nextHop[MAX_NODES][MAX_NODES];
int i, j, k;
for (i = 0; i < nodes; i++) { for (j = 0; j < nodes; j++) {
distance[i][j] = costMatrix[i][j];
if (costMatrix[i][j] == INFINITY || i == j)nextHop[i][j] = -1;
else
nextHop[i][j] = j;
}
}
for (k = 0; k < nodes; k++) {for (i = 0; i < nodes; i++) {
for (j = 0; j < nodes; j++) {
if (distance[i][j] > distance[i][k] + distance[k][j]) {distance[i][j] = distance[i][k] + distance[k][j];
nextHop[i][j] = nextHop[i][k];
}
}
}
}
for (i = 0; i < nodes; i++) {
printf("\nRouting table for node%d:\n", i + 1);printf("Destination\tCost\tNext Hop\n");
for (j = 0; j < nodes; j++) {if (i != j) {
printf("%d\t\t%d\t%d\n", j + 1, distance[i][j], nextHop[i][j] + 1);
}
}
}
}
int main() {
int nodes = MAX_NODES;
int costMatrix[MAX_NODES][MAX_NODES] = {
{0, 2, 4, INFINITY},
{2, 0, 1, 7},
{4, 1, 0, 3},
{INFINITY, 7, 3, 0}
```

```
};
distanceVector(costMatrix, nodes);
return 0;
}
```

EXPECTED OUTPUT:

```
rajashekhar-sunkara@MiWiFi-R4A-srv:~/Documents$ gcc djikstra\'s.c -o djikstra
rajashekhar-sunkara@MiWiFi-R4A-srv:~/Documents$ ./djikstra

Routing table for node 1:
Destination     Cost    Next Hop
2               2       2
3               3       2
4               6       2

Routing table for node 2:
Destination     Cost    Next Hop
1               2       1
3               1       3
4               4       3

Routing table for node 3:
Destination     Cost    Next Hop
1               3       2
2               1       2
4               3       4

Routing table for node 4:
Destination     Cost    Next Hop
1               6       3
2               4       3
3               3       3
```

EXERCISE-12

AIM: Write a Program to implement Broadcast tree by taking subnet of hosts.

DESCRIPTION: A Broadcast Tree is a structure that allows data to be sent from one host to all other hosts in a network efficiently. In a broadcast network, a single packet can be sentto multiple nodes simultaneously. The program will create a simple representation of a subnet using an adjacency matrix, and it will simulate the broadcasting of messages froma specified source node to allother nodes in the subnet.

PROGRAM:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_HOSTS 5

void broadcastTree(int graph[MAX_HOSTS][MAX_HOSTS], int hosts, int source) {
    bool visited[MAX_HOSTS] = {false};
    int queue[MAX_HOSTS];
    int front = -1, rear = -1;

    // Start with the source host
    visited[source] = true;
    queue[++rear] = source;

    printf("Broadcasting message from Host %d:\n", source + 1);

    while (front != rear) {
        front++;
        int currentHost = queue[front];
        int i;

        for (i = 0; i < hosts; i++) {
            if (graph[currentHost][i] == 1 && !visited[i]) {
                visited[i] = true;
                queue[++rear] = i;
                printf("Message sent to Host %d\n", i + 1);
            }
        }
    }
}

int main() {
    int graph[MAX_HOSTS][MAX_HOSTS] = {
        {0, 1, 1, 0, 0},
        {1, 0, 0, 1, 0},
        {1, 0, 0, 0, 1},
        {0, 1, 0, 0, 0},
        {0, 0, 1, 0, 0}
```

```
    };

    int sourceHost = 0; // Change this to any host you want to broadcast from
    broadcastTree(graph, MAX_HOSTS, sourceHost);

    return 0;
}
```

Expected Output:

```
rajashekhar-sunkara@MiWiFi-R4A-srv:~/Documents$ gcc djikstra\'s.c -o djikstra
rajashekhar-sunkara@MiWiFi-R4A-srv:~/Documents$ ./djikstra


Broadcasting message from Host 1:
Message sent to Host 2
Message sent to Host 3
Message sent to Host 4
Message sent to Host 5
```

AIM 13: Wireshark

What is Wireshark?

Wireshark is an open-source packet analyzer, which is used for **education, analysis, software development, communication protocol development, and network troubleshooting**.

It is used to track the packets so that each one is filtered to meet our specific needs. It is commonly called as a **sniffer, network protocol analyzer, and network analyzer**. It is also used by network security engineers to examine security problems.

Wireshark is a free to use application which is used to apprehend the data back and forth. It is often called as a free packet sniffer computer application. It puts the network card into an unselective mode, i.e., to accept all the packets which it receives.

Uses of Wireshark:

Wireshark can be used in the following ways:

1. It is used by network security engineers to examine security problems.
2. It allows the users to watch all the traffic being passed over the network.
3. It is used by network engineers to troubleshoot network issues.
4. It also helps to troubleshoot latency issues and malicious activities on your network.
5. It can also analyze dropped packets.
6. It helps us to know how all the devices like laptop, mobile phones, desktop, switch, routers, etc., communicate in a local network or the rest of the world.

Functionality of Wireshark:

Wireshark is similar to tcpdump in networking. **Tcpdump** is a common packet analyzer which allows the user to display other packets and TCP/IP packets, being transmitted and received over a network attached to the computer.

It has a graphic end and some sorting and filtering functions. Wireshark users can see all the traffic passing through the network.

Wireshark can also monitor the unicast traffic which is not sent to the network's MAC address interface. But, the switch does not pass all the traffic to the port.

Hence, the promiscuous mode is not sufficient to see all the traffic. The various network taps or **port mirroring** is used to extend capture at any point.

Port mirroring is a method to monitor network traffic. When it is enabled, the switch sends the copies of all the network packets present at one port to another port.

Features of Wireshark

- It is multi-platform software, i.e., it can run on Linux, Windows, OS X, FreeBSD, NetBSD, etc.
- It is a standard three-pane packet browser.
- It performs deep inspection of the hundreds of protocols.
- It often involves live analysis, i.e., from the different types of the network like the Ethernet, loopback, etc., we can read live data.
- It has sort and filter options which makes ease to the user to view the data.
- It is also useful in VoIP analysis.
- It can also capture raw USB traffic.
- Various settings, like timers and filters, can be used to filter the output.
- It can only capture packet on the PCAP (an application programming interface used to capture the network) supported networks.
- Wireshark supports a variety of well-documented capture file formats such as the PcapNg and Libpcap. These formats are used for storing the captured data.
- It is the no.1 piece of software for its purpose. It has countless applications ranging from the **tracing down, unauthorized traffic, firewall settings, etc**.

Installation of Wireshark Software

Below are the steps to install the Wireshark software on the computer:

- Open the web browser.
- Search for '**Download Wireshark**.'
- Select the Windows installer according to your system configuration, either 32-bt or 64-bit. Save the program and close the browser.
- Now, open the software, and follow the install instruction by accepting the license.
- The Wireshark is ready for use.
- On the network and Internet settings option, we can check the interface connected to our computer.

I. **Capture Using Wire shark**

Use Wireshark to perform **Packet** a packet capture of network traffic:

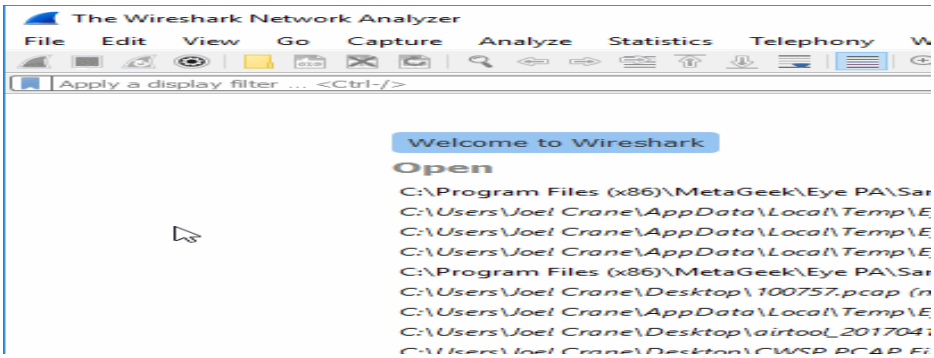
Visit the Wireshark download site, and download the appropriate Wireshark package or installer for the operating system running on the system which is to be used for packet capture.

When installing, ensure all components are selected for installation, including the optional “Winpcap” application.

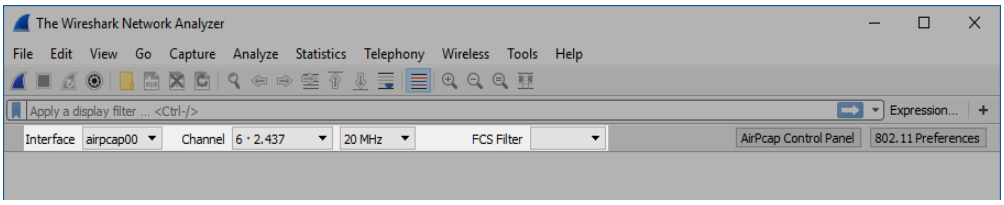
In some cases, you may want to perform packet captures with Wireshark. One case might be when you want to perform a packet capture on channel 12 or 13.

Set up the Packet Capture

1. Click **View** > **Wireless Toolbar.** The Wireless Toolbar will appear just below the Main toolbar.



2. Use the **Wireless Toolbar** to configure the desired channel and channel width.

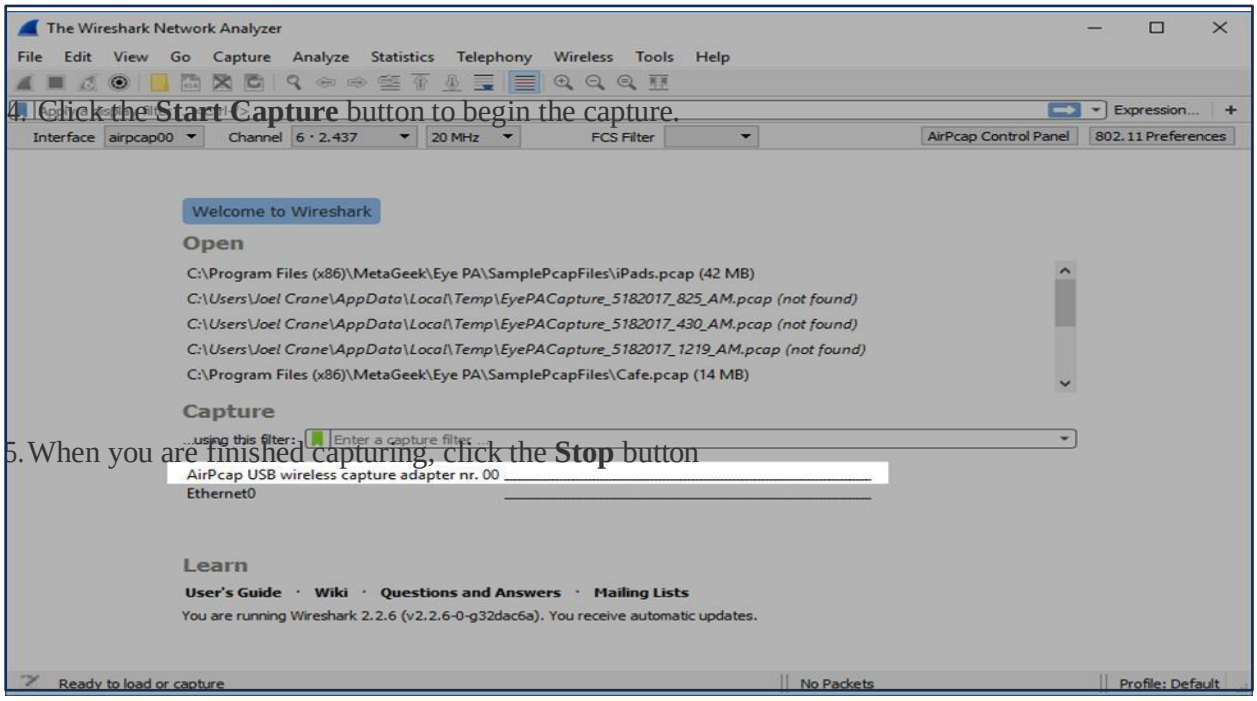


3. Under **Capture**, click on **AirPcap USB wireless capture adapter** to select the capture interface.

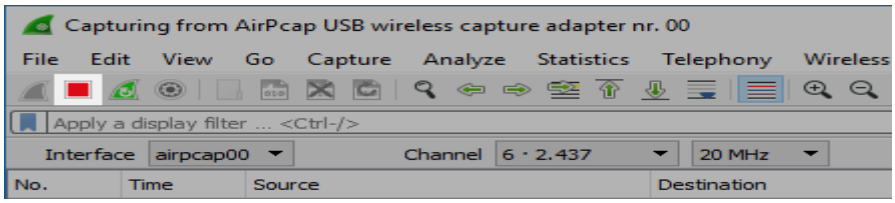
*Note: If the AirPcap isn't listed, press **F5** to refresh the list of available packet capture interfaces.*

Note: The AirPcap has been discontinued by RiverBed and is 802.11n only.

4. Click the **Start Capture** button to begin the capture.

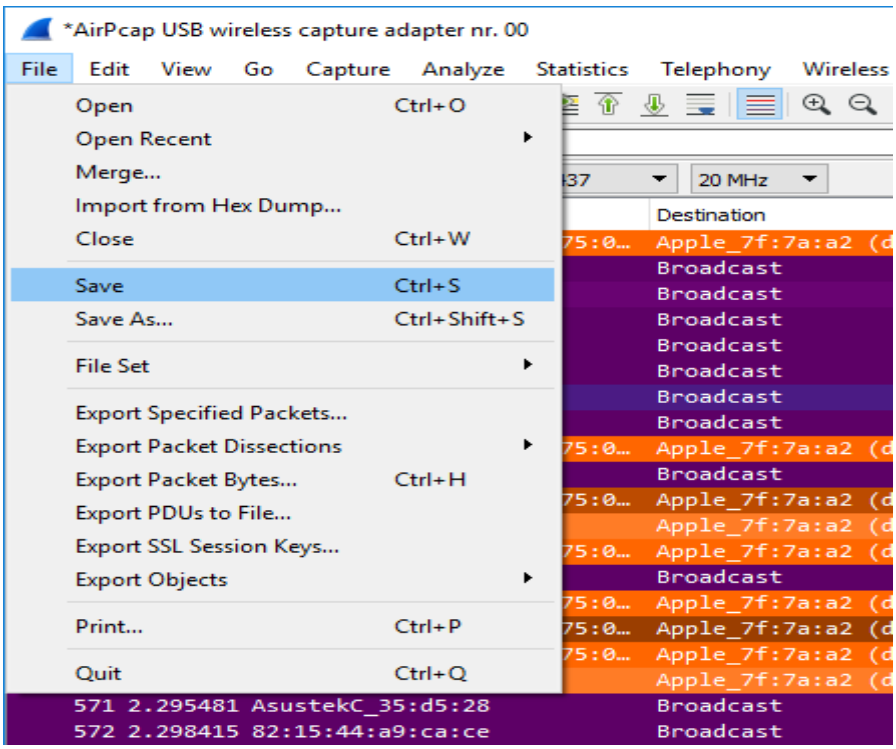


5. When you are finished capturing, click the **Stop** button



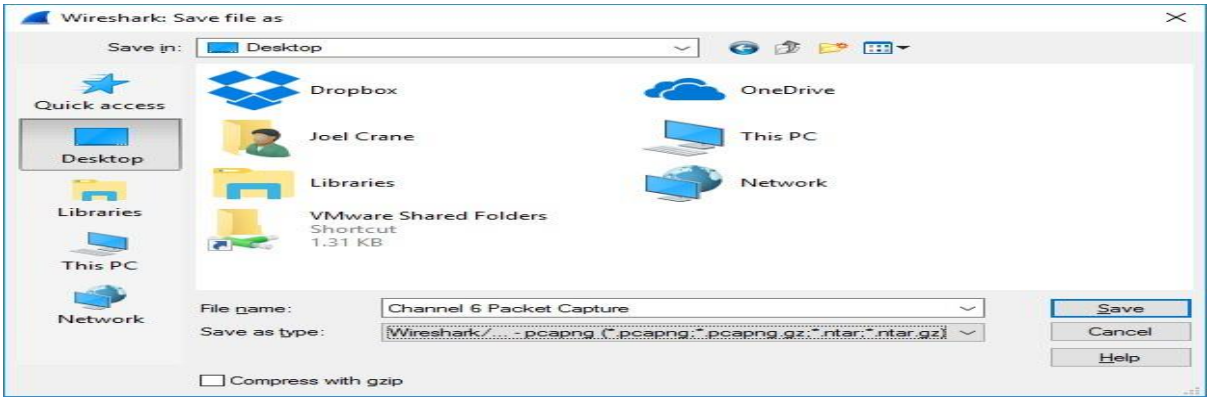
Saving the Capture

1. To save the capture, click **File** > **Save**.



2. Name the file, and click **Save**.

*Note: **.Pcap** and **.Pcap-ng** are good filetypes to use for the capture if you plan to use Eye P.A. to open the capture.*



2. Eye P.A. can now open the capture file

I. STARTING WIRESHARK:

Two different methods for starting Wireshark are available. These include the Start menu and the Run command box

Method 1 - Start Menu

To start Wireshark using the Start menu:

1. Open the **Start** menu.
2. Select **All Programs**.
3. Select **Wireshark**.

Method 2 - Run Command

To start Wireshark using the Run command box:

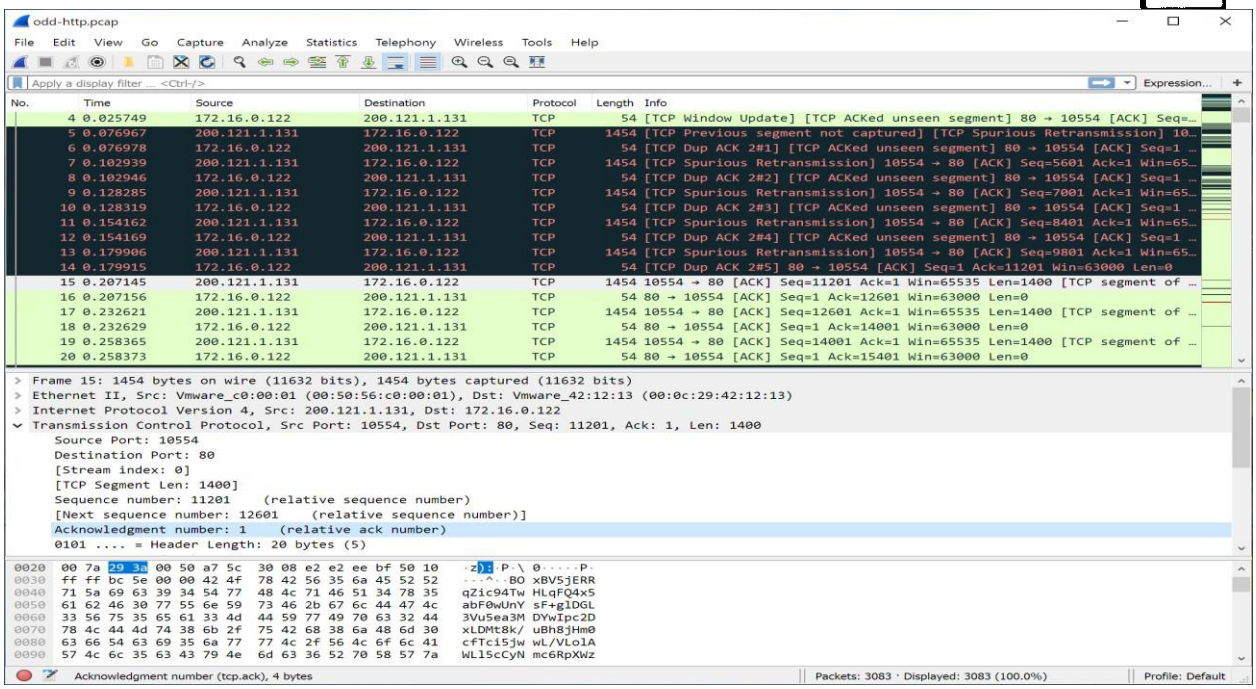
1. Open the **Start** menu or press the **Windows key + R**.
2. Type **Wireshark** in the Run command box.
3. Press **Enter**

III. Viewing Packets You Have Captured:

Once you have captured some packets or you have opened a previously saved capture file, you can view the packets that are displayed in the packet list pane by simply clicking on a packet in the packet list pane, which will bring up the selected packet in the tree view and byte view panes.

You can then expand any part of the tree to view detailed information about each protocol in each packet. Clicking on an item in the tree will highlight the corresponding bytes in the byte view. An example with a TCP packet selected is shown in Figure 6.1, "Wireshark with a TCP packet selected for viewing". It also has the Acknowledgment number in the TCP header selected, which shows up in the byte view as the selected bytes.

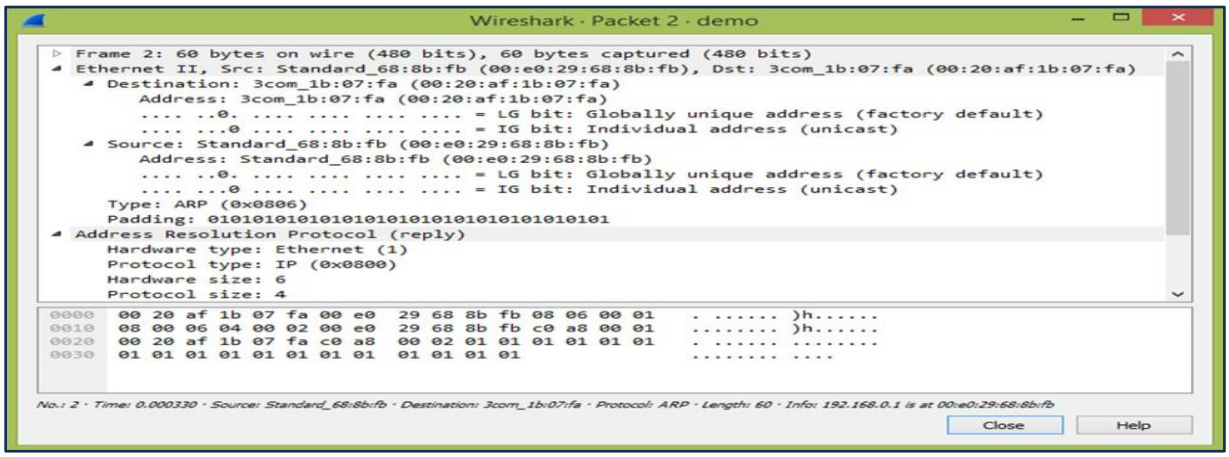
Figure 6.1. Wireshark with a TCP packet selected for viewing



You can also select and view packets the same way while Wireshark is capturing if you selected “Update list of packets in real time” in the “Capture Preferences” dialog box.

In addition you can view individual packets in a separate window as shown in Figure 6.2, “Viewing a packet in a separate window”. You can do this by double-clicking on an item in the packet list or by selecting the packet in which you are interested in the packet list pane and selecting View → Show Packet in New Window. This allows you to easily compare two or more packets, even across multiple files.

Figure 6.2. Viewing a packet in a separate window



Along with double-clicking the packet list and using the main menu there are a number of other ways to open a new packet window:

- Hold down the shift key and double-click on a frame link in the packet details.

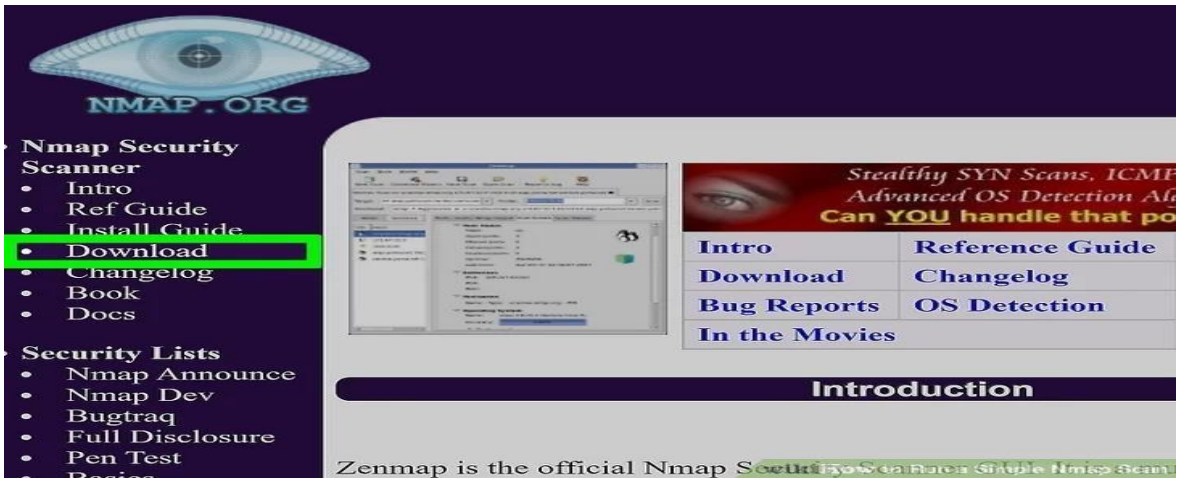
AIM 14: How to run Nmap scan

PROCEDURE:

NMAP: NMAP stands for Network Mapper which is an open-source tool used for network exploration and security auditing, in comparison to this, a tool named **Nessus** is used by industry professionals. These tools are mainly used by cybersecurity experts and hackers.

2. Nmap is a network scanner created by Gordon Lyon.
3. Nmap is used to discover hosts and services on a computer network by sending packets and analyzing the responses.
4. Nmap provides a number of features for probing computer networks, including host discovery and service and operating system detection
5. Ensuring that your router is protected from unwanted intruders is one of the foundations of a secure network.
6. One of the basic tools for this job is **Nmap**, or Network Mapper.
7. This program will scan a target and report which ports are open and which are closed, among other things. Security specialists use this program to test the security of a network.

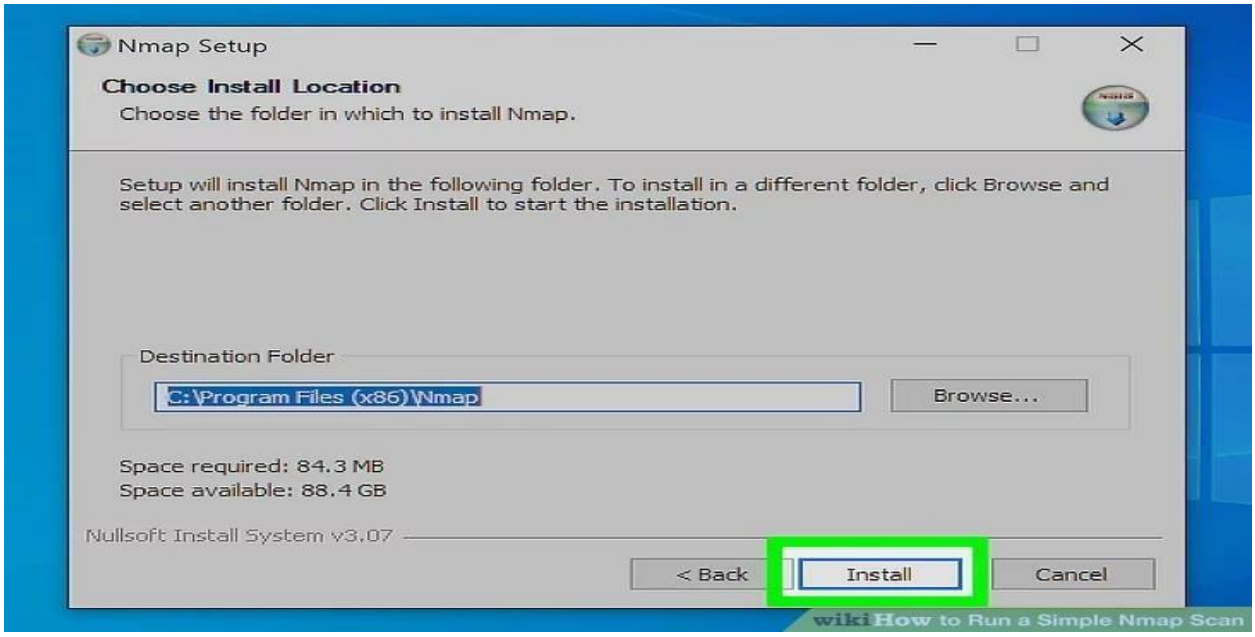
METHOD 1:



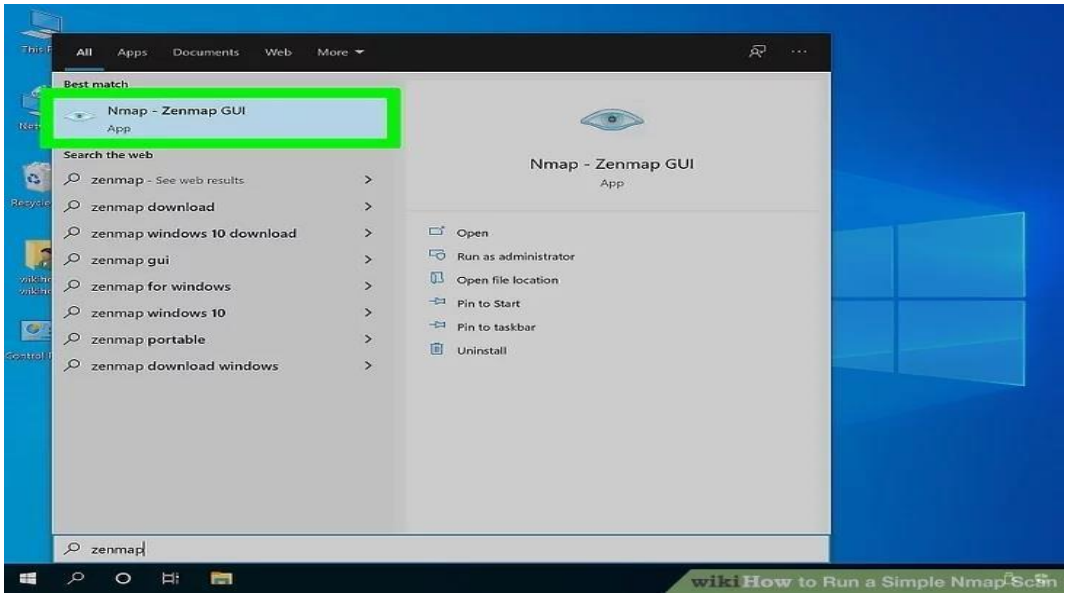
1. Download the Nmap installer. This can be found for free from the developer's website. It is highly recommended that you download directly from the developer to avoid any potential viruses or fake files. Downloading the Nmap installer includes Zenmap, the graphical interface for Nmap which makes it easy for newcomers to perform scans without having to learn command lines.

The Zenmap program is available for Windows, Linux, and Mac OS X. You can find the installation files for all operating systems on the Nmap website.

2.Install Nmap. Run the installer once it is finished downloading. You will be asked which components you would like to install. In order to get the full benefit of Nmap, keep all of these checked. Nmap will not install any adware or spyware.



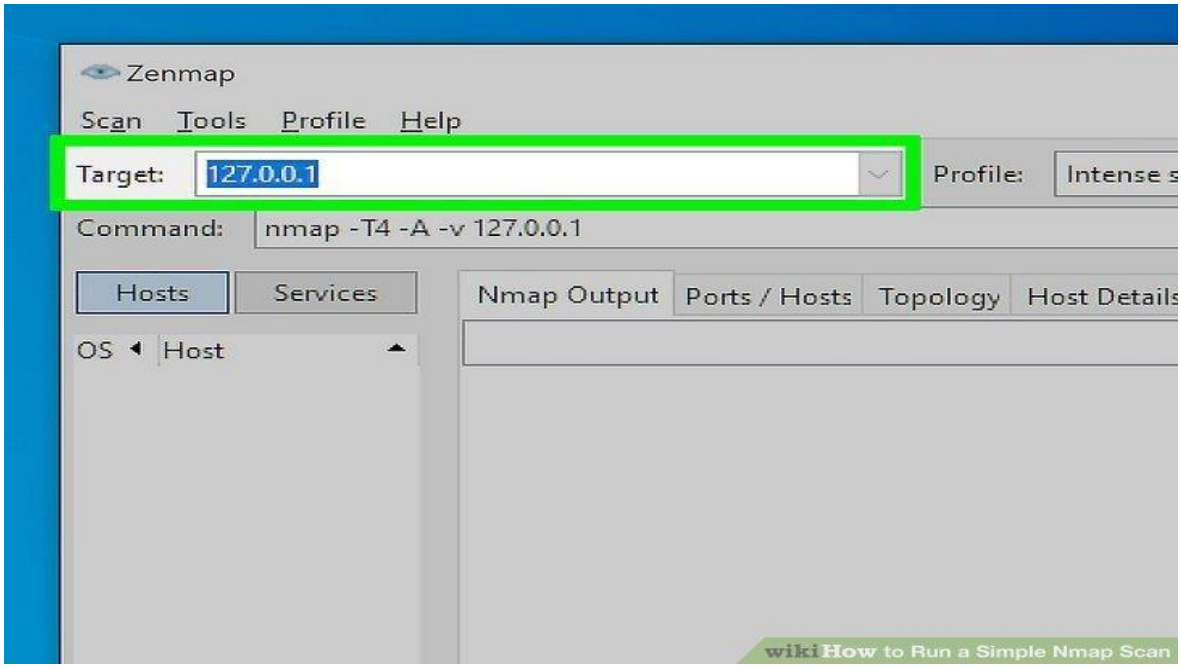
3. **Run the "Nmap – Zenmap" GUI program.** If you left your settings at default during installation, you should be able to see an icon for it on your desktop. If not, look in your Start menu. Opening Zenmap will start the program.



Enter in the target for your scan : The Zenmap program makes scanning a fairly simple process. The first step to running a scan is choosing your target. You can enter a domain (example.com), an IP address (127.0.0.1), a network (192.168.1.0/24), or a combination of those.

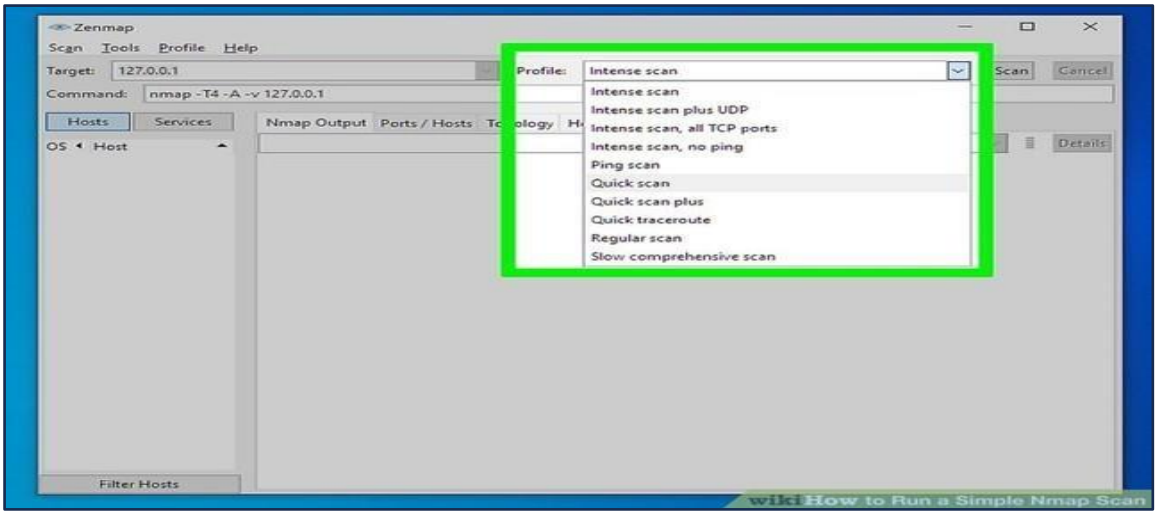
Depending on the intensity and target of your scan, running an Nmap scan may be against the terms of your internet service provider, and may land you in hot water. Always check

- your local laws and your ISP contract before performing Nmap scans on targets other than your own network.

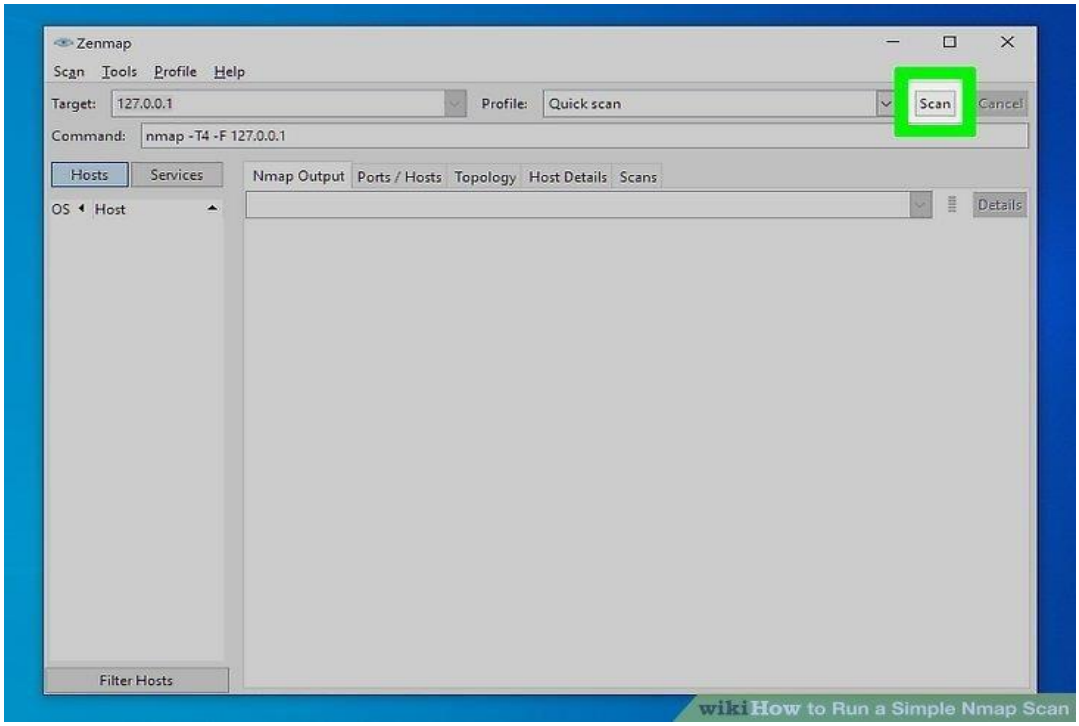


4. **Choose your Profile.** Profiles are preset groupings of modifiers that change what is scanned. The profiles allow you to quickly select different types of scans without having to type in the modifiers on the command line. Choose the profile that best fits your needs:[1]

- **Intense scan** - A comprehensive scan. Contains Operating System (OS) detection, version detection, script scanning, traceroute, and has aggressive scan timing. This is considered an intrusive scan.
- **Ping scan** - This scan simply detects if the targets are online, it does not scan any ports.
- **Quick scan** - This is quicker than a regular scan due to aggressive timing and only scanning select ports.
- **Regular scan** - This is the standard Nmap scan without any modifiers. It will return ping and return open ports on the target.

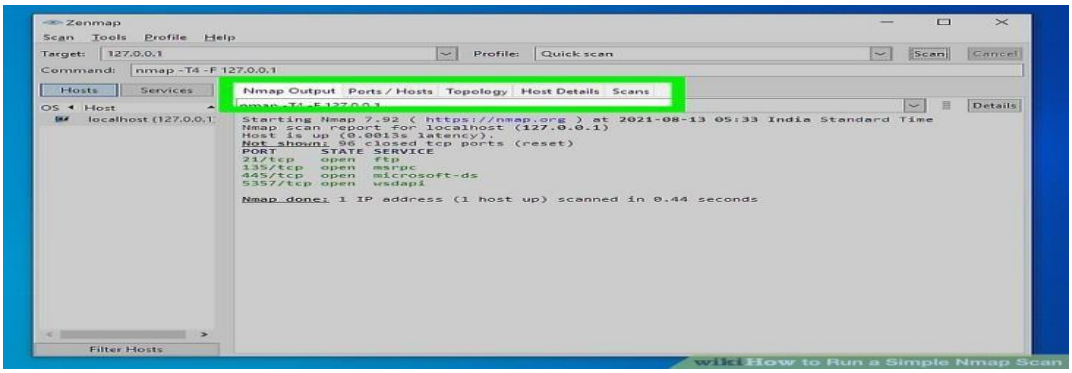


6. **Click Scan to start scanning.** The active results of the scan will be displayed in the Nmap Output tab. The time the scan takes will depend on the scan profile you chose, the physical distance to the target, and the target's network configuration.



7. **Read your results.** Once the scan is finished, you'll see the message "Nmap done" at the bottom of the Nmap Output tab. You can now check your results, depending on the type of scan you performed. All of the results will be listed in the main Nmap Output tab, but you can use the other tabs to get a better look at specific data.[2]

- **Ports/Hosts** - This tab will show the results of your port scan, including the services for those ports.
- **Topology** - This shows the traceroute for the scan you performed. You can see how many hops your data goes through to reach the target.
- **Host Details** - This shows a summary of your target learned through scans, such as the number of ports, IP addresses, hostnames, operating systems, and more.
- **Scans** - This tab stores the commands of your previously-run scans. This allows you to quickly re-scan with a specific set of parameters.



AIM 15: Operating System Detection using Nmap

OS detection using NMAP:

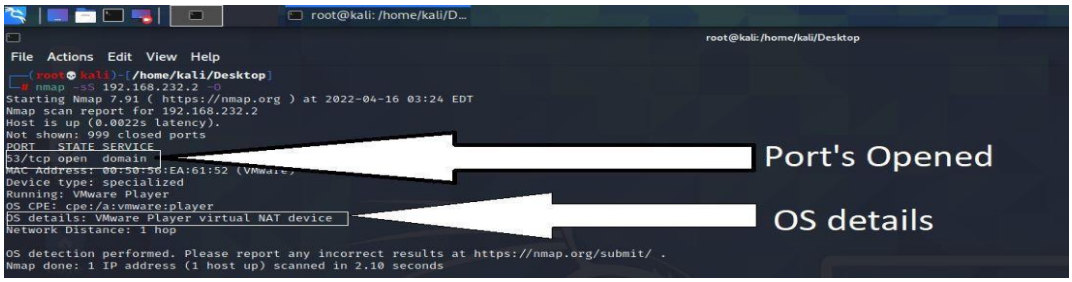
Now we need to run the actual commands to perform OS detection using NMAP, and at first, we will get the IP address of the host system, and then will perform a scan to get all active devices on the network.

Step 1: Getting the IP of the System
Ifconfig

Step 2: List of active devices in the Network
nmap -sn 192.168.232.128/24

Let's do an SYN scan with OS detection in one of the active IPs

Let's select IP: 192.168.232.2
nmap -sS 192.168.232.2 –O



Running: VMware Player.

OS details: VMware Player virtual NAT device.

Let's now perform an Aggressive scan To guess the OS

- **-sV** stands for Service version.
- **-A** stands for Aggressive.

It will only display the chance of Operation System (OS) on the host computer with the help of Probability and Percentage.

nmap -sV 192.168.232.2 –A

AIM 16: Do the following using NS2 Simulator

I. NS2 Simulator-Introduction

Network Simulator (Version 2), widely known as NS2, is simply an event-driven simulation tool that has proved useful in studying the dynamic nature of commu-nication networks. Simulation of wired as well as wireless network functions and protocols (e.g., routing algorithms, TCP, UDP) can be done using NS2. In general, NS2 provides users with a way of specifying such network protocols and simulating their corresponding behaviors.

Due to its flexibility and modular nature, NS2 has gained constant popularityin the networking research community since its birth in 1989. Ever since, several revolutions and revisions have marked the growing maturity of the tool, thanks to substantial contributions from the players in the field. Among these are the University of California and Cornell University who developed the REAL networksimulator,[1] the foundation on which NS is invented. Since 1995 the Defense Advanced Research Projects Agency (DARPA) supported the development of NS through the Virtual InterNetwork Testbed (VINT) project Currently the National Science Foundation (NSF) has joined the ride in development. Last but not the least, the group of researchers and developers in the community are constantly working to keep NS2 strong and versatile.

Again, the main objective of this book is to provide the readers with insights into the NS2 architecture. This chapter gives a brief introduction to NS2. NS2 Beginners are recommended to go thorough the detailed introductory online resources. For example, NS2 official website provides NS2 source code as well as detailed installation instruction. The web pages in and are among highly recommended ones which provide tutorial and examples for setting up basic NS2 simulation.

2.1 Basic Architecture

Figure 2.1 shows the basic architecture of NS2. NS2 provides users with an executable command "ns" which takes one input argument, the name of a Tcl simulation scripting file. In most cases, a simulation trace file is created and is used to plot graph and/or to create animation.

NS2 consists of two key languages: C and Object-oriented Tool Command Language (OTcl). While the C defines the internal mechanism (i.e., a backend)of the simulation, the OTcl sets up simulation by assembling and configuring the objects as well as scheduling discrete events (i.e., a frontend). The C and the OTcl are linked together using TclCL. Mapped to a C object, variables inthe OTcl domains are sometimes referred to as *handle*s. Conceptually, a handleis just a string (e.g., "_o10") in the OTcl domain and does not contain any functionality. Instead, the functionality (e.g., receiving a packet) is defined in the mapped C object (e.g., of class Connector). In the OTcl domain, a handle acts as a frontend which interacts with users and other OTcl objects.

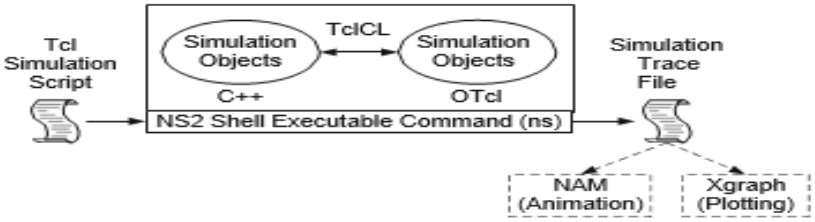
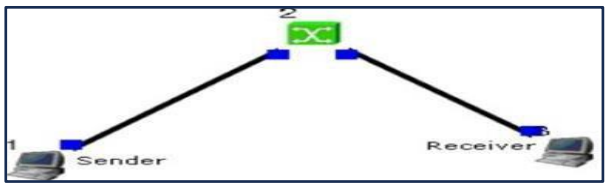


Fig. 2.1 Basic architecture of NS

II. Simulate to Find the Number ofPackets Dropped

Simulate a three-node point-to-point network with a duplex link between them. Set the queue size and vary the bandwidth and find the number of packets dropped.



Sender:-

stcp –p 2000 –l 1024 1.0.1.2

Receiver:-

rtcp –p 2000 –l 1024

Parameters:-

Drop Packets and Collision Packets.

Step1: Drawing topology

1. Select/click the HOST icon on the toolbar and click the left mouse button on the editor,to place a HOST1 on the editor.

   Repeat the above procedure and place another host "HOST2" on the editor.

2. Select/click the HUB icon on the toolbar and click the left mouse button on the editor,to place HUB1 on the editor.

3. Click on the LINK icon on the toolbar and connect HOST1 to HUB1 and HUB1 toHOST2
4. Click on the “E” icon on the toolbar to save the current topology
   e.g: file1.tpl
   (Look for the ******.tpl extension.)

NOTE: Changes cannot / (should not) be done after selecting the “E” icon.

Step2: Configuration

1. Double click the left mouse button while cursor is on HOST1 to open the HOSTwindow.
2. Select Add button on the HOST window to invoke the command window and provide the following command in the command textbox.
   stg –u 1024 100 1.0.1.2
3. Click OK button on the command window to exit and once again click on the OKbutton on the HOST window to exit.
4. Double click the left mouse button while cursor is on HOST2 to open the HOSTwindow.
5. Select Add button on the HOST window to invoke the command window andprovide the following command in the command textbox.
   rtg –u –w log1
6. Click OK button on the command window to exit.
7.
8. Click NODE EDITOR Button on the HOST window and select the MAC tab from themodal window that pops up.
9. Select LOG STATISTICS and select checkboxes for Number of Drop Packet andNumber of Collisions in the MAC window
10. Click OK button on the MAC window to exit and once again click on the OK buttonon the HOST window to exit.

Note: To set QUEUE size

1. Double click the left mouse button while cursor is on HOST2 to open the HOSTwindow.
2. Click NODE EDITOR Button on the HOST window and select the FIFO tab from themodal window that pops up.
3. Click OK button on the FIFO window to exit and once again click on the OK buttonon the HOST window to exit.

Step3: Simulate

i. Click "R" icon on the tool bar

ii. Select Simulation in the menu bar and click/ select RUN in thedropdown list to execute the simulation.

iii. To start playback select "►" icon located at the bottom right corner ofthe editor.

iv. To view results, Open up new TERMINAL window, move to file1.results folder and open collision and drop log files in separate TERMINAL window.

Caution: file1 is the hypothetical name given to this simulation.

Changing configurations

Change 1

1. Open the above file,
2. Do not change the topology or any other configuration,
3. Select E icon on the toolbar
4. Reduce the bandwidth at link2 by double clicking the left mouse button while cursor ison link2 .(Change bandwidth on both tabs Uplink/Downlink)
5. Repeat Step3 (Simulate)

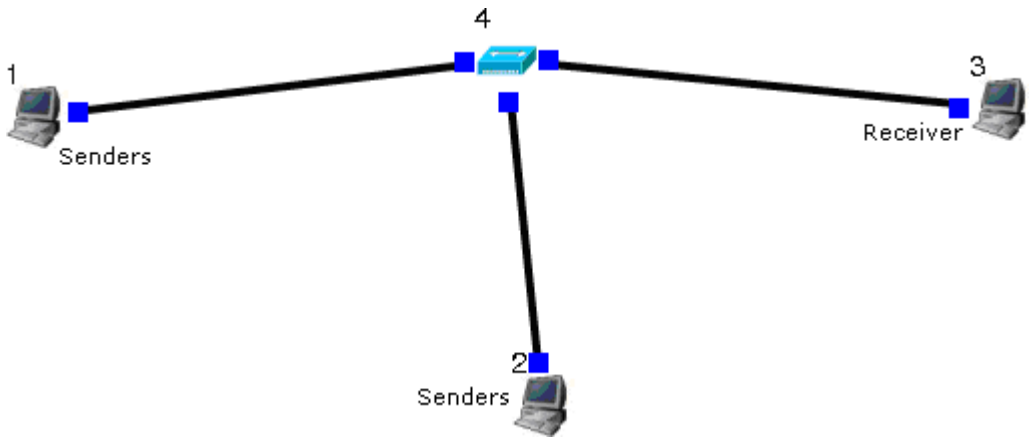
Change 2

1. Open the above file,
2. Remove HUB and replace it with SWITCH.
3. Do not change anything in the configurationRepeat Step3(Simulate)

II. Simulate to Find the Number of Packets Dropped by TCP/UDP

Simulate a four-node point-to-point network and connect the link as follows: Apply a TCP agent between n0 to n3 and apply a UDP agent between n1 and n3. Apply relevant applications over TCP and UDP agents changing the parameters and determine the numberof packets sent by two agents.

Topology:-



Sender:-

stcp –p 3000 –l 1024 1.0.1.3
stg –u 1024 1.0.1.3

Receiver:-

rtcp –p 3000 –l 1024
rtg –u 3000

Parameters:-

Throughput of incoming and outgoing Packets

Step1: Drawing topology

1. Select/click the HOST icon on the toolbar and click the left mouse button on the editor,to place a host on the editor.
   Repeat the above procedure and place two other hosts "HOST2" and "HOST3"on the editor.

2. Select/click the HUB (or SWITCH) icon on the toolbar and click the left mousebutton on the editor, to place a HUB (or SWITCH) on the editor.
3. Click on the LINK icon on the toolbar and connect HOST1 to HUB, HOST2 to HUB andHUB to HOST3
4. Click on the "E" icon on the toolbar to save the current topology **e.g:** file2.tpl(Look for the ******.tpl extension.)

NOTE: Changes cannot / (should not) be done after selecting the "E" icon.

Step2: Configuration

1. Double click the left mouse button while cursor is on HOST1 to open the HOSTwindow.

2. Select Add button on the HOST window to invoke the command window and provide the following command in the command textbox.
   stcp –p 21 –l 1024 1.0.1.3

3. Click OK button on the command window to exit

4. Click NODE EDITOR Button on the HOST window and select the MAC tab from themodal window that pops up.

5. Select LOG STATISTICS and select checkbox for output throughput in the MACwindow

6. Click OK button on the MAC window to exit and once again click on the OK buttonon the HOST window to exit.

7. Double click the left mouse button while cursor is on HOST2 to open the HOSTwindow.

8. Select Add button on the HOST window to invoke the command window and provide the following command in the command textbox.

   stg –u 1024 100 1.0.1.3

9. Click OK button on the command window to exit

10. Click NODE EDITOR Button on the HOST window and select the MAC tab from themodal window that pops up.

11. Select LOG STATISTICS and select checkbox for output throughput in the MACwindow

12. Double click the left mouse button while cursor is on HOST3 to open the HOSTwindow.

13. Select Add button on the HOST window to invoke the command window andprovide the following command in the command textbox.

    rtcp –p 21 –l 1024

14. Click OK button on the command window to exit.

15. Also add the following command on HOST3

    rtg –u –w log1

16. Click NODE EDITOR Button on the HOST window and select the MAC tab from themodal window that pops up.

17. Select LOG STATISTICS and select checkbox for input and outputthroughput in the MAC window

18. Click OK button on the MAC window to exit and once again click on the OK buttonon the HOST window to exit.

Step3: Simulate

i. Click “R” icon on the tool bar

ii. Select Simulation in the menu bar and click/ select RUN in thedropdown list to execute the simulation.

iii. To start playback select "►" icon located at the bottom right corner of the editor.

iv. To view results, Open up new TERMINAL window, move to file2.results folder and open input and output throughput log files in separate TERMINALwindow.

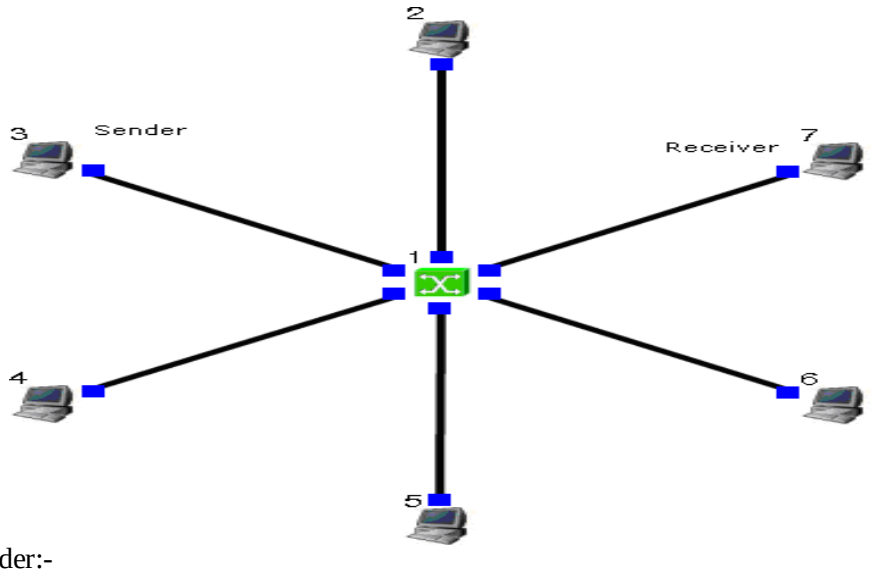
Caution: file2 is the hypothetical name given to this simulation.

(Refer Step 1.4)

III. Simulate to Find the Number of PacketsDropped due to Congestion

Simulate the transmission of ping messages over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.

Topology:-



Sender:-

stcp –p 2000 –l 1024 1.0.1.4

Receiver:-

rtcp –p 2000 –l 1024

Command Console:-

Goto tools-> simulation time and change Simulation time to 100. During run mode, double click host 2 and then click command console. And execute the following command.

ping

1.0.1.4

Paramete

rs:-

Drop Packets and Collision Packets.

Step1: Drawing topology

1. Select/click the SUBNET icon on the toolbar and click the left mouse button on theeditor, to place a SUBNET on the editor.

2. A pop up window appears requesting the number of nodes and radius for the subnetSet number of nodes=6;
   Set radius of subnet >150
3. Click on the "E" icon on the toolbar to save the current topology **e.g:** file4.tpl(Look for the ******.tpl extension.)

NOTE: Changes cannot / (should not) be done after selecting

Step2: Configuration

4. Double click the left mouse button while cursor is on a HOST to open the HOSTwindow.

5. Click NODE EDITOR Button on the HOST window and select theINTERFACE tab (1st tab) from the modal window that pops up.

6. Determine the IP address of the selected host.

7. Click OK button on the INTERFACE window to exit and once again click on the OKbutton on the HOST window to exit.

8. Repeat the above step for 2 other HOSTS

9. Also click NODE EDITOR Button on the HOST window and select the MAC tabfrom the modal window that pops up.

10. Select LOG STATISTICS and select checkbox for drop and collision log statistics inthe MAC window

11. Click OK button on the MAC window to exit and once again click on the OK buttonon the HOST window to exit.

12. Repeat steps 6 to 9 for the other hosts selected at step 5.

13. Select G_Setting from the menu bar and select Simulation from the drop down listSet simulation time>600sec

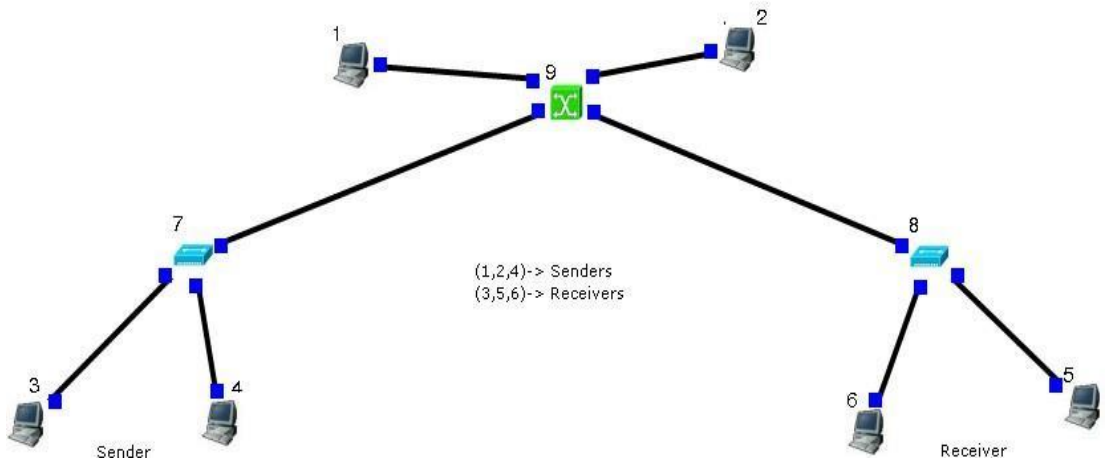
Step3: Simulate

i. Click “R” icon on the tool bar
ii. Select Simulation in the menu bar and click/ select RUN in the dropdown list toexecute the simulation.
iii. During simulation, open a new terminal window.
iv. Type **ping IP address** of a host in the subnet at the command prompt.
v. To view results, Open up new TERMINAL window, move to file4.results folder andopen drop and collision log files in separate TERMINAL window.

V.Simulate to Compare Data Rate & Throughput

Simulate an Ethernet LAN using N nodes (6-10), change error rate and data rate and compare throughput.

Topology:-



Sender:-
stcp –p 2000 –l 1024 1.0.1.4

Receiver:-
rtcp –p 2000 –l 1024

Double click on receiver link and change BER to 0.000001, Run

Again.Parameters:-
Throughput of outgoing Packets

Step1: Drawing topology

1. Select/click the HOST icon on the toolbar and click the left mouse button on the editor,to place HOST1 on the editor.
   Repeat the above procedure and place 5 other hosts “HOST2”, “HOST3”, “HOST4”, “HOST5”, and “HOST6”on the editor.

2. Select/click the HUB icon on the toolbar and click the left mouse button on the editor,to place HUB1 on the editor.
   Repeat the above procedure and place another host “HUB2” on the editor

3. Select/click the SWITCH icon on the toolbar and click the left mouse button on theeditor, to place SWITCH1 on the editor.

4. Click on the LINK icon on the toolbar and connect HUB1 to SWITCH1 and HUB2 toSWITCH1.

5. Click on the "E" icon on the toolbar to save the current topology **e.g:** file5.tpl (Look for the ******.tpl extension.)

NOTE: Changes cannot / (should not) be done after selecting the "E" icon.

Step2: Configuration

1. Double click the left mouse button while cursor is on HOST1 to open the HOSTwindow.Select Add button on the HOST window to invoke the command window andprovide the following command in the command textbox.

   stcp –p 21 –l 1024 1.0.1.4

2. Click OK button on the command window to exit and once again click on the OKbutton on the HOST window to exit.

3. Repeat this step at HOST 2 and HOST3, but use different commands

   stcp –p 21 –l 1024 1.0.1.5 at HOST2
   stcp –p 21 –l 1024 1.0.1.6 at HOST3

4. Double click the left mouse button while cursor is on HOST4 to open the HOSTwindow.

5. Select Add button on the HOST window to invoke the command window and provide the following command in the command textbox.

6. Click OK button on the command window to exit.

7. Click NODE EDITOR Button on the HOST window and select the MAC from

8. Select LOG STATISTICS and select checkbox for output throughput in the MACwindow

□ Program 1: RSA Algorithm for Secure Data Transmission

Aim:

Write a program to implement the RSA algorithm for encryption and decryption.

Algorithm Steps:

1. Choose two prime numbers $p$and q.
2. Compute $n = p \times q$.
3. Compute Euler's totient function $\phi(n) = (p-1)(q-1)$.
4. Choose an encryption key $e$such that $1 < e < \phi(n)$and $gcd(e, \phi(n)) = 1$.
5. Determine decryption key $d$such that $d \times e \bmod \phi(n) = 1$.
6. Encrypt message: $C = M^e \bmod n$.
7. Decrypt message: $M = C^d \bmod n$.

RSA Algorithm Implementation

Program:

```
#include <stdio.h>
#include <math.h>

long int p, q, n, t, e, d;
long int temp[100], encrypted[100];
char msg[100];

int isPrime(long int num) {
    int i;
    for (i = 2; i <= sqrt(num); i++) {
        if (num % i == 0)
            return 0;
    }
    return 1;
}
```

```
long int gcd(long int a, long int b) {
    if (a == 0)
        return b;
    return gcd(b % a, a);
}

long int modInverse(long int e, long int phi) {
    long int d = 0;
    long int k = 1;
    while (1) {
        k = k + phi;
        if (k % e == 0)
            return (k / e);
    }
}

long int modPower(long int base, long int exp, long int mod) {
    long int result = 1;
    for (int i = 0; i < exp; i++) {
        result = (result * base) % mod;
    }
    return result;
}

int main() {
    long int phi;
    printf("Enter two prime numbers (p and q): ");
    scanf("%ld %ld", &p, &q);
```

```
if (!isPrime(p) || !isPrime(q)) {
    printf("Both numbers must be prime.\n");
    return 0;
}

n = p * q;
phi = (p - 1) * (q - 1);

printf("Enter public key e (coprime to %ld): ", phi);
scanf("%ld", &e);

if (gcd(e, phi) != 1) {
    printf("e is not coprime to %ld\n", phi);
    return 0;
}

d = modInverse(e, phi);

printf("\nPublic Key: (%ld, %ld)", e, n);
printf("\nPrivate Key: (%ld, %ld)\n", d, n);

printf("\nEnter message (as integer): ");
long int msg;
scanf("%ld", &msg);

long int cipher = modPower(msg, e, n);
long int decrypted = modPower(cipher, d, n);
```

```
    printf("\nEncrypted Message: %ld", cipher);
    printf("\nDecrypted Message: %ld\n", decrypted);

    return 0;
}
```

Sample Output:

Enter two prime numbers (p and q): 7 17

Enter public key e (coprime to 96): 5

Public Key: (5, 119)

Private Key: (77, 119)

Enter message (as integer): 12

Encrypted Message: 3

Decrypted Message: 12

Program 2: Subnetting and IP Address Calculation

Aim:

Write a program to calculate subnet mask, number of hosts, and network address for a given IP and subnet.

Algorithm Steps:

1. Input IP address and CIDR (e.g. /27).
2. Convert IP to binary form.
3. Determine subnet mask using CIDR bits.
4. Calculate number of hosts = $2^{(32-CIDR)} - 2$.
5. Compute network address using bitwise AND between IP and subnet mask.

Program:

```
#include <stdio.h>
#include <math.h>

int main() {
    int ip[4], cidr;
    int mask[4], network[4];
    int i, host_bits;

    printf("Enter IP address (e.g., 192.168.10.25): ");
    scanf("%d.%d.%d.%d", &ip[0], &ip[1], &ip[2], &ip[3]);

    printf("Enter CIDR value (e.g., 27): ");
    scanf("%d", &cidr);

    host_bits = 32 - cidr;

    int total_hosts = pow(2, host_bits) - 2;

    // Calculate subnet mask
    for (i = 0; i < 4; i++) {
        if (cidr >= 8) {
            mask[i] = 255;
            cidr -= 8;
        } else {
            mask[i] = (int)(256 - pow(2, 8 - cidr));
            cidr = 0;
        }
    }
```

```
    // Calculate network address
    for (i = 0; i < 4; i++) {
        network[i] = ip[i] & mask[i];
    }

    printf("\nSubnet Mask: %d.%d.%d.%d", mask[0], mask[1], mask[2], mask[3]);
    printf("\nNetwork Address: %d.%d.%d.%d", network[0], network[1], network[2],
network[3]);
    printf("\nTotal Hosts per Subnet: %d\n", total_hosts);

    return 0;
}
```

Sample Output:

Enter IP address (e.g., 192.168.10.25): 192.168.10.25

Enter CIDR value (e.g., 27): 27

Subnet Mask: 255.255.255.224

Network Address: 192.168.10.0

Total Hosts per Subnet: 30